

Untitled Project

Software Architecture Document

Official technical blueprint

Version	1.0
Status	Approved
Date	2026-09-18
Author	Architect AI (AI Dev Team)
Generated by	AI Dev Team for Visual Studio Code

This document was produced by the Architect AI from the approved requirements and defines what should be built. Coding tasks are derived from it by the Prompt Engineer AI; deviations require an architecture revision.

Document information

Metadata for this architecture document. Versions follow MAJOR.MINOR semantics: MINOR for refinements, MAJOR for structural changes.

Property	Value
Project	Untitled Project
Document	Software Architecture Document
Version	1.0
Status	Approved
Date	2026-09-18
Author	Architect AI (AI Dev Team)
Approver	Collaborator AI on behalf of the user

Revision history

Version	Date	Status	Changes
1.0	2026-09-18	Approved	Initial architecture

How to read this document

Sections are numbered hierarchically. Diagrams are numbered as figures and placed directly after the paragraph that introduces them. Tables summarise requirements, interfaces and decisions. Code examples are illustrative, not implementations. Architecture decisions record context, decision, alternatives and consequences.

Contents

1 Project overview	8
1.1 Source of requirements and assumptions	8
1.2 Scope boundaries	8
2 Project objectives	10
2.1 Primary objectives	10
2.2 Secondary objectives	10
2.3 Non-objectives	11
3 Requirements	12
3.1 Functional requirements	12
3.2 Non-functional requirements	12
3.3 Constraints	13
4 System architecture	14
4.1 Architectural style	14
4.2 Layers and components	14
4.3 Motion architecture	14
4.4 Theme architecture	15
4.5 Interaction flow	15
5 Architecture diagrams	16
5.1 Component structure	16
5.2 Page structure and user flow	16
5.3 Lightbox state machine	17
5.4 Bootstrap sequence	18
5.5 Artwork data model	19
6 AI-agent workflow	21
6.1 Roles and responsibilities for this project	21
6.2 Workflow	21
6.3 Hand-off rules	21
6.4 Escalation to the Architect	21
6.5 Verification tooling available to the team	22
7 Technology stack	23
7.1 Selected technologies	23
7.2 Rejected alternatives	23
7.3 Browser support matrix	23
7.4 Performance-relevant choices	24

8 Folder/project structure	25
8.1 Workspace layout	25
8.2 Internal structure of index.html	25
8.3 Naming conventions	25
8.4 Content editing surface	26
9 Frontend architecture	27
9.1 Markup layer	27
9.2 Style layer	27
9.3 Behaviour layer	28
9.4 Responsive strategy	28
10 Backend architecture	29
10.1 Responsibility mapping	29
10.2 Hosting model	29
10.3 Future extension points	29
11 Database architecture	30
11.1 The ARTWORKS record	30
11.2 Validation rules	30
11.3 Relationships	30
11.4 Content lifecycle	30
12 API architecture	32
12.1 Internal module contracts	32
12.2 Shared motion helper	32
12.3 Event contracts	32
12.4 External requests	33
13 Data-flow diagrams	34
13.1 Content flow at load	34
13.2 Tilt and sheen flow	34
13.3 Pointer light and lift flow	35
13.4 Reveal flow	35
13.5 Capability gating	35
14 User-flow diagrams	36
14.1 Fine-pointer visitor	36
14.2 Touch visitor	38
14.3 Keyboard visitor	38

14.4 Reduced-motion visitor	39
14.5 Focus and announcement rules	39
15 AI communication architecture	40
15.1 Message artefacts	40
15.2 Communication flow per segment	40
15.3 Shared vocabulary	41
15.4 Context the Coder must always receive	41
15.5 Evidence standards for the Tester	41
16 Security architecture	42
16.1 Threat assessment	42
16.2 Content Security Policy (CSP)	42
16.3 Secure-by-construction rules for the Coder	43
16.4 Verification	43
17 Authentication architecture	44
17.1 Decision	44
17.2 Why no authentication is correct here	44
17.3 Trust boundaries	44
17.4 Conditions that would reintroduce authentication	44
18 Deployment architecture	45
18.1 Deployment targets	45
18.2 Runtime dependencies at deploy time	45
18.3 Release procedure	45
18.4 Caching and performance headers	45
18.5 Content update deployment	46
19 Development workflow	47
19.1 Local environment	47
19.2 Segment cycle	47
19.3 Branching and commits	49
19.4 Coding conventions	49
19.5 Definition of done for a segment	49
20 Testing strategy	50
20.1 Test levels	50
20.2 Constraint check specification	50
20.3 Behavioural test cases (excerpt)	50

20.4 Performance test protocol	51
20.5 Test artefacts	51
21 Debugging strategy	52
21.1 Diagnostic order	52
21.2 Built-in debug aids	54
21.3 DevTools techniques by symptom	54
21.4 Reproduction discipline	54
21.5 Escalation boundary	54
22 Error-handling strategy	56
22.1 Error categories and handling	56
22.2 Bootstrap error containment	56
22.3 Guarantees	57
22.4 What is deliberately not handled	57
23 Future scalability	58
23.1 Growth dimensions	58
23.2 Pre-defined seams	58
23.3 Performance ceiling	58
23.4 Signals that a MAJOR revision is due	59
24 Development phases	61
24.1 Phase overview	61
24.2 Relative effort	61
24.3 Phase dependencies and checkpoints	61
24.4 Checkpoint contents	62
25 Implementation roadmap	63
25.1 Segment details	63
25.2 Requirement coverage	64
26 Architecture decisions	65
26.1 ADR-1: Treat the original idea as the requirements baseline	65
26.2 ADR-2: Single self-contained HTML file with region markers	65
26.3 ADR-3: CSS-custom-property-driven motion on the compositor	65
26.4 ADR-4: Spring interpolation in vanilla JS rather than a library	65
26.5 ADR-5: Native <dialog> for the lightbox	66
26.6 ADR-6: Inline SVG placeholders generated from a palette seed	66
26.7 ADR-7: Progressive enhancement with capability gating	66

26.8 ADR-8: No backend, mailto: contact	66
26.9 ADR-9: Manual, checklist-driven testing plus a mechanical constraint check	66
26.10 ADR-10: Motion phase before interaction phase	67
27 Assumptions and limitations	68
27.1 Assumptions	68
27.2 Limitations	68
27.3 Open questions for the Collaborator	69

1 Project overview

This document describes the technical architecture for a single-page artist portfolio website. The site presents an artist's name and statement, a grid of nine to twelve artworks with a lightbox viewer, an "about" section and a contact section. The distinguishing quality of the project is its visual and motion design: a high-gloss, print-like aesthetic (deep neutral background, one accent colour, high-contrast display serif paired with a clean grotesque) combined with tasteful, cursor-reactive motion — tilting artwork tiles with a moving specular sheen, a pointer-following soft light source, and scroll-triggered fade-and-rise section entries.

The technical constraints are deliberately strict: the entire deliverable is **one self-contained HTML file** containing inline CSS and vanilla JavaScript. There is no framework, no bundler and no build step. Theming is implemented with CSS custom properties. All animation must run at 60 fps using only `transform` and `opacity`, must be spring-damped and eased (cubic-bezier, roughly 400 ms), must degrade gracefully on touch devices and must be fully disabled under `prefers-reduced-motion`. The site must be fully responsive and keyboard navigable.

1.1 Source of requirements and assumptions

The upstream Requirements Summary, Refined Requirements and Research Report supplied to the Architect are empty. The Summarizer explicitly reported that it could not proceed. This architecture is therefore derived **directly from the user's original idea** recorded in Project Memory, which is detailed and internally consistent enough to serve as the requirements baseline. The following assumptions are made explicit and must be confirmed by the Collaborator:

#	Assumption	Rationale	Impact if wrong
A1	The original idea in Project Memory is the authoritative requirements source	Upstream deliverables are empty	Requirements section must be regenerated
A2	Placeholder images may be generated in-file (inline SVG / CSS) rather than external files	"Self-contained HTML file" and "placeholder images are fine"	Would require external asset folder
A3	No backend, no form submission endpoint; contact section uses mailto: and social links	Idea specifies no server-side components	A form handler service would need selecting
A4	Modern evergreen browsers (last two versions of Chrome, Firefox, Safari, Edge) are the target	"Modern, minimal build", use of IntersectionObserver and CSS custom properties	Polyfills or fallbacks would be needed
A5	Hosting is static (GitHub Pages, Netlify or any file host)	Single HTML file	None significant
A6	Artwork content will be swapped in later by editing a data array in the file	Placeholders are explicitly accepted	A CMS would change the architecture materially

1.2 Scope boundaries

In scope: page structure, theme system, typography, glass card treatment, tilt/sheen effect, pointer light, scroll reveal, lightbox, responsive layout, touch degradation, reduced-motion handling, keyboard accessibility, performance discipline, and the placeholder content system.

Out of scope: content management system, server-side code, analytics, contact form backend, multi-page navigation, internationalisation, image optimisation pipeline, and automated deployment infrastructure beyond copying one file to a static host.

2 Project objectives

The objectives translate the idea into measurable goals that the Coder, Tester and Debugger can verify. Each objective has an acceptance criterion so that "done" is unambiguous.

2.1 Primary objectives

ID	Objective	Acceptance criterion
O1	Deliver a single self-contained HTML file	index.html opens from disk with no network dependencies except optional web fonts; zero external JS/CSS files
O2	Achieve a glossy, editorial visual identity	Design tokens (background, accent, type scale, letter-spacing) implemented as CSS custom properties; card glass treatment uses inner highlight, low-opacity border and restrained shadow; no multi-stop decorative gradients
O3	Provide tasteful cursor-reactive motion	Tile tilt "d 8° on each axis; sheen position tracks pointer; pointer light lifts nearby cards; all eased with cubic-bezier and spring damping "H 400 ms
O4	Reveal sections on scroll	Each section fades and rises once when entering the viewport, driven by IntersectionObserver
O5	Maintain 60 fps	Only transform and opacity animate; no layout-triggering properties in animation paths; DevTools performance recording shows no long frames during pointer movement
O6	Respect user preferences	With prefers-reduced-motion: reduce, tilt, sheen, pointer light and reveal animations are disabled; content is fully visible without motion
O7	Be fully responsive and touch-friendly	Layout works from 320 px to 2560 px wide; on touch/coarse pointers, tilt is disabled and cards remain static with hover-equivalent focus states
O8	Be keyboard navigable	All tiles, lightbox controls and links reachable via Tab; lightbox opens with Enter/Space, closes with Escape, traps focus while open and restores focus on close
O9	Present intentional placeholder content	9–12 artworks with realistic titles, media, dimensions and years; artist statement and about copy read as genuine prose, not lorem ipsum

2.2 Secondary objectives

- **Maintainability:** all content lives in one clearly labelled data array so the artist can swap in real works without touching layout code.
- **Theming:** switching from near-black to bone-white variant requires changing one `data-theme` attribute or a small token block.
- **Progressive enhancement:** the page is readable and navigable with JavaScript disabled; JS only adds motion and the lightbox.
- **Accessibility baseline:** semantic landmarks, alt text on every artwork, visible focus rings that fit the aesthetic, sufficient colour contrast (WCAG AA — Web Content Accessibility Guidelines level AA) for body text.

2.3 Non-objectives

The project does not aim for SEO (Search Engine Optimisation) tooling, server rendering, analytics, or content editing UI. These are explicitly excluded to protect the "no framework bloat" constraint.

3 Requirements

This section restates the requirements as a structured, numbered list. Functional requirements (FR) describe behaviour; non-functional requirements (NFR) describe qualities. Priorities follow MoSCoW (Must, Should, Could, Won't).

3.1 Functional requirements

ID	Requirement	Priority	Verification
FR-01	Hero section displays artist name in display serif and a one-line statement	Must	Visual inspection, DOM check
FR-02	Work grid shows 9–12 artwork tiles rendered from a data array	Must	Count tiles in DOM
FR-03	Clicking or activating a tile opens a lightbox with enlarged image, title, medium, dimensions, year	Must	Manual + keyboard test
FR-04	Lightbox supports previous/next navigation, Escape to close, click-outside to close	Should	Manual test
FR-05	About section with artist bio and portrait placeholder	Must	Visual inspection
FR-06	Contact section with email (mailto:) and social links	Must	Link check
FR-07	Artwork tiles tilt toward the cursor (max ~8°) with a moving specular sheen	Must	Pointer test, computed transform check
FR-08	A soft light source follows the pointer and lifts cards it is near	Must	Visual inspection
FR-09	Sections fade and rise on entering the viewport via IntersectionObserver	Must	Scroll test
FR-10	Tilt, sheen and pointer light are disabled on coarse pointers (touch)	Must	Device emulation
FR-11	All motion disabled under prefers-reduced-motion: reduce	Must	Emulate media feature in DevTools
FR-12	Theme switchable between dark (near-black) and light (bone white) via one attribute	Should	Toggle data-theme
FR-13	Placeholder artwork images generated in-file (inline SVG or CSS) with distinct, intentional appearance	Should	Visual inspection
FR-14	Sticky or minimal top navigation with anchor links to sections	Could	Manual test

3.2 Non-functional requirements

ID	Requirement	Priority	Verification
NFR-01	Single HTML file, no build step, no external JS/CSS	Must	File inspection
NFR-02	Vanilla JS only; no libraries	Must	Code review
NFR-03	Animations use only transform and opacity	Must	Code review, DevTools Layers/Rendering

ID	Requirement	Priority	Verification
NFR-04	60 fps during pointer motion and scroll on a mid-range laptop	Must	Performance recording
NFR-05	Pointer handlers throttled to requestAnimationFrame; no per-event layout reads inside hot loops	Must	Code review
NFR-06	Responsive from 320 px to 2560 px; grid adapts 1 !' 2 !' 3 columns	Must	Viewport emulation
NFR-07	Keyboard navigable; visible focus; lightbox focus trap and restore	Must	Keyboard walkthrough
NFR-08	WCAG AA contrast for body text against background	Should	Contrast checker
NFR-09	Page usable without JavaScript (content visible, links work)	Should	Disable JS
NFR-10	Total file size under 150 KB excluding web fonts	Could	File size check
NFR-11	Web fonts loaded with font-display: swap and system fallbacks	Should	Network throttling test

3.3 Constraints

- **C1:** No framework, bundler, transpiler or package manager.
- **C2:** CSS custom properties are the theming mechanism.
- **C3:** Easing is cubic-bezier, approximately 400 ms, spring-damped in JS-driven effects.
- **C4:** No 2013-style decorative gradients; glass treatment is subtle.

4 System architecture

Although the deliverable is a single file, it is internally organised as a small layered system so that the Coder can work section by section and the Tester can verify each layer independently. There is no server, database or network API; the "backend" is a static data array in the file.

4.1 Architectural style

The architecture is **progressive enhancement over a static document**. The HTML layer is complete and meaningful on its own. The CSS layer provides the visual identity and the static end-states of all animations. The JavaScript layer is a set of small, independent *enhancement modules* that attach behaviour to elements marked with `data-*` attributes. Each module guards itself against unsupported conditions (coarse pointer, reduced motion, missing API) and does nothing if conditions are not met.

4.2 Layers and components

Layer	Component	Responsibility
Document	Semantic HTML skeleton	Landmarks (header, main, section, footer), hero, grid container, about, contact, lightbox dialog
Data	ARTWORKS array	9–12 artwork records (id, title, medium, dimensions, year, palette seed, alt)
Style	Design tokens	CSS custom properties for colour, type scale, spacing, radius, motion durations, easing curves
Style	Layout & typography	Grid, responsive breakpoints, fluid type scale via clamp()
Style	Glass card treatment	Inner highlight, low-opacity border, restrained shadow, sheen layer
Behaviour	Capabilities	Detects prefers-reduced-motion, pointer: coarse, IntersectionObserver support; exposes flags
Behaviour	Renderer	Builds tiles and placeholder SVGs from ARTWORKS into the grid
Behaviour	Tilt	Per-tile pointer tracking, spring-damped rotation "d 8°, sheen position via CSS variables
Behaviour	PointerLight	Single fixed-position radial light element following the pointer; proximity-based card lift
Behaviour	Reveal	IntersectionObserver-driven is-visible class toggling for fade-and-rise
Behaviour	Lightbox	Open/close/navigate, focus trap and restore, Escape/arrow key handling
Behaviour	Bootstrap	Reads capabilities, initialises modules in order, single entry point

4.3 Motion architecture

All JS-driven motion follows one pattern: the pointer event writes a *target* value; a single `requestAnimationFrame` loop interpolates the *current* value toward the target using a spring (stiffness/damping) or exponential smoothing; the loop writes CSS custom properties (`--rx`, `--ry`, `--sx`, `--sy`) on the element; CSS `transform` and the sheen `background-position/transform` consume those variables. This keeps the browser on the compositor path and confines DOM writes to one place. The loop self-suspends when all values have settled, so idle CPU cost is zero.

4.4 Theme architecture

Tokens are declared on `:root` (dark default) and overridden under `[data-theme="light"]`. Components reference only tokens, never literal colours. The single accent colour is one token (`--accent`) so the artist can retune the palette in one line.

4.5 Interaction flow

The diagram below shows how a pointer movement travels through the system without triggering layout.

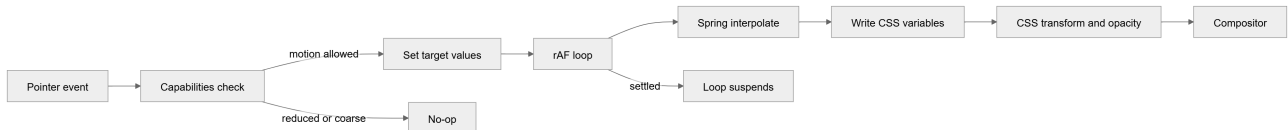


Figure 1 — System architecture

5 Architecture diagrams

This section collects the structural diagrams that define the system. Each diagram is referenced by the Coder when implementing the corresponding part and by the Tester when deriving test cases.

5.1 Component structure

The single file is composed of clearly delimited blocks. The diagram shows the static composition and which modules depend on which.

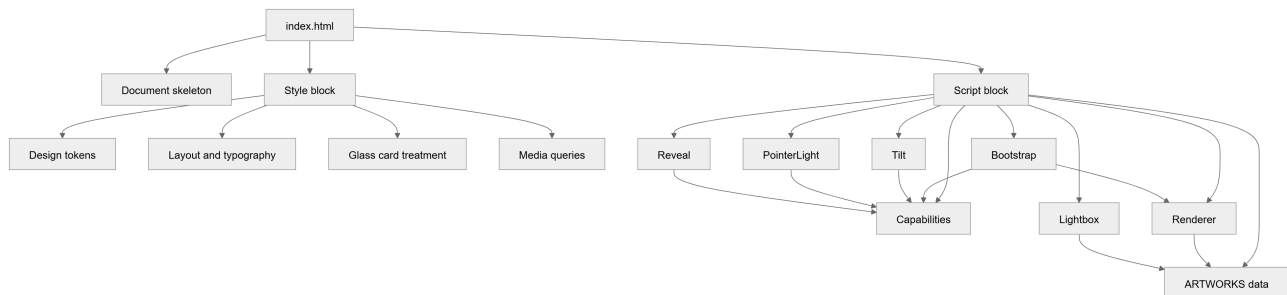
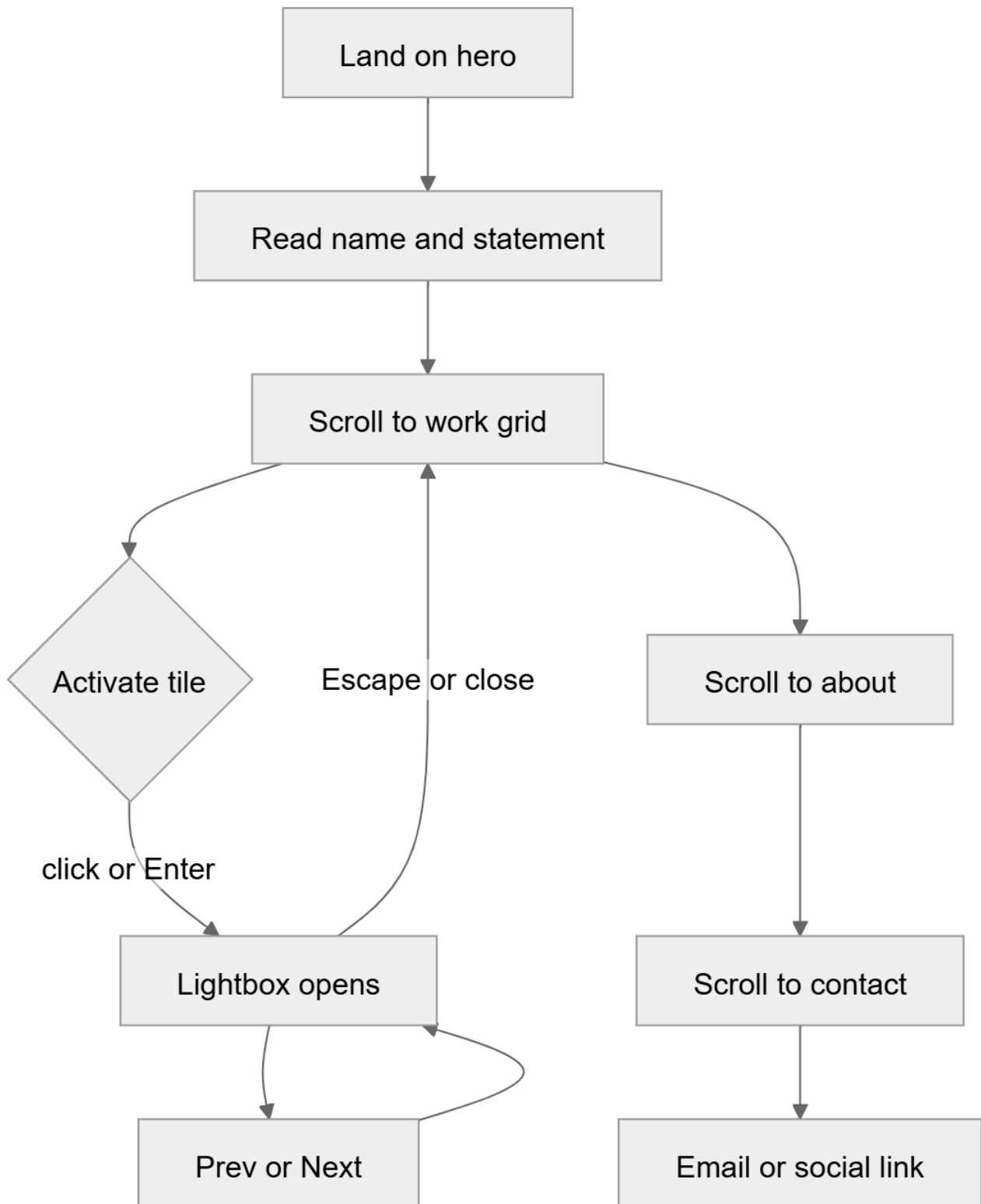


Figure 2 — Architecture diagrams

5.2 Page structure and user flow

Visitors move top-to-bottom through four sections; the lightbox is a modal branch from the work grid.

*Figure 3 — Architecture diagrams*

5.3 Lightbox state machine

The lightbox is the only stateful UI component with multiple transitions, so its states are specified explicitly.

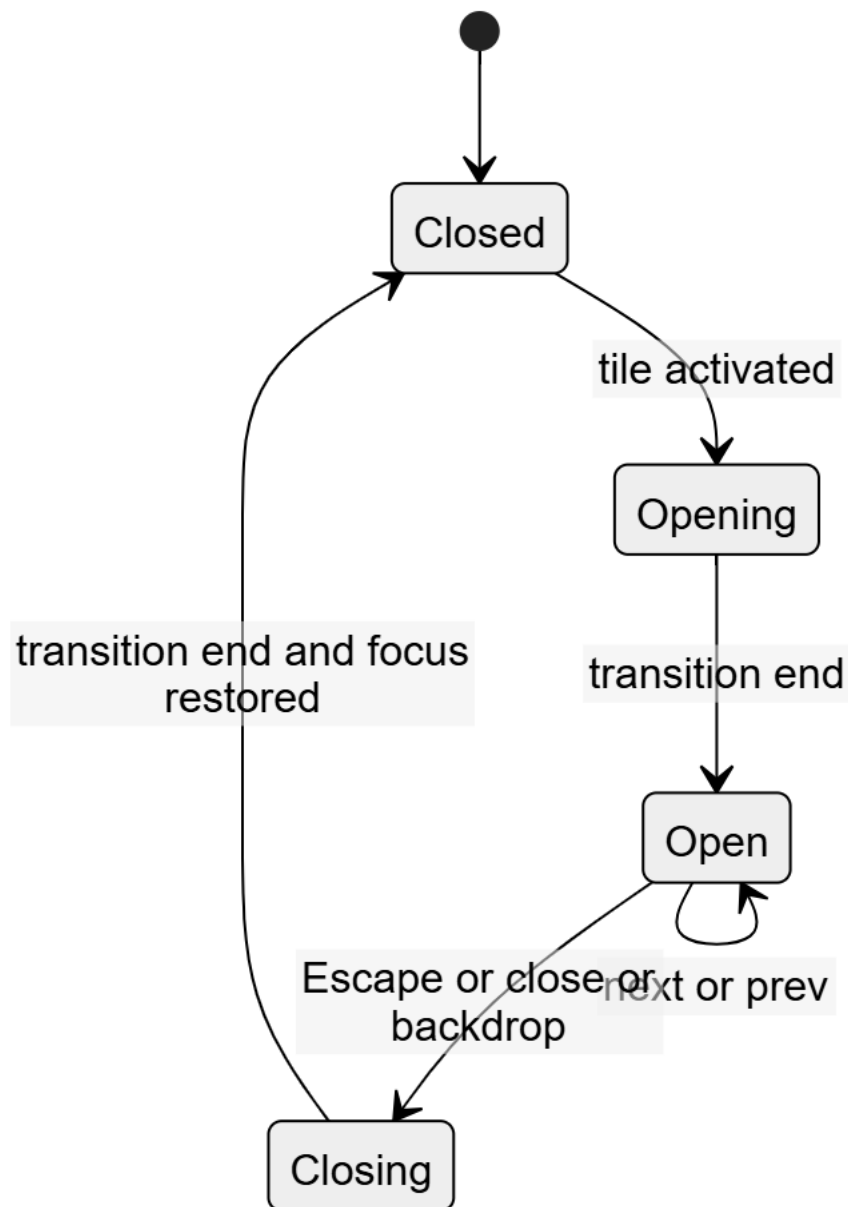


Figure 4 — Architecture diagrams

5.4 Bootstrap sequence

The initialisation order matters: capabilities must be known before any motion module attaches listeners, and tiles must exist before Tilt and Reveal observe them.

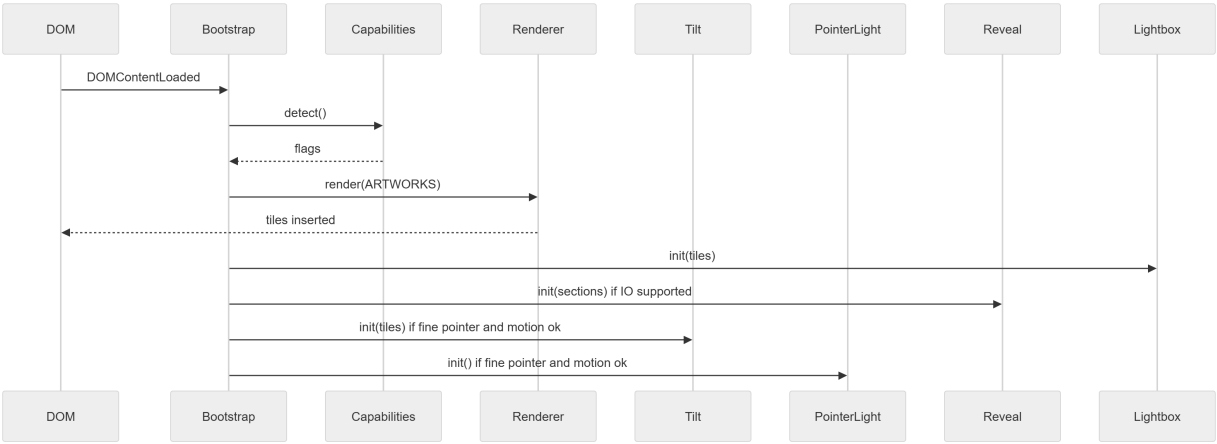


Figure 5 — Architecture diagrams

5.5 Artwork data model

There is no database, but the artwork record shape is the contract between data, renderer and lightbox.

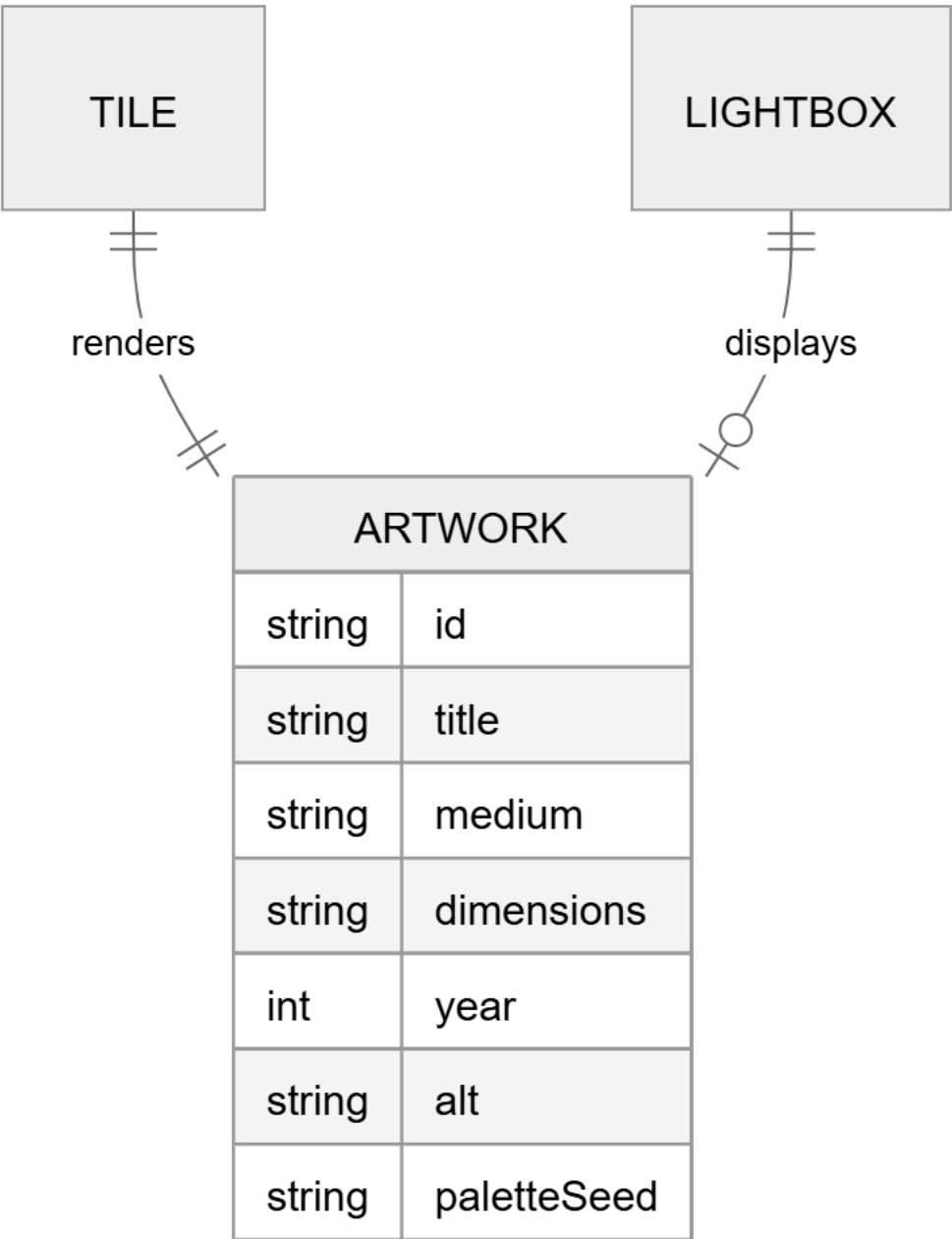


Figure 6 — Architecture diagrams

6 AI-agent workflow

The project itself contains no AI components. This section therefore describes how the AI Dev Team will build and verify this project, including hand-off points and escalation paths back to the Architect.

6.1 Roles and responsibilities for this project

Role	Responsibility on this project	Hand-off artefact
Collaborator	Confirms assumptions A1–A6, approves architecture, sequences development segments	Approval notes, segment order
Prompt Engineer	Turns each development segment into a precise Coder prompt referencing this document's section IDs and requirement IDs	Coder prompt per segment
Coder	Implements the segment inside index.html without introducing external files or libraries	Updated index.html, change notes
Tester	Verifies requirement IDs mapped to the segment using the manual and scripted checks in the testing strategy	Test report (pass/fail per ID)
Debugger	Fixes failures reported by the Tester; may not change architecture	Patched index.html, root-cause note
Summarizer	Records the completed segment in Project Memory	Development chapter

6.2 Workflow

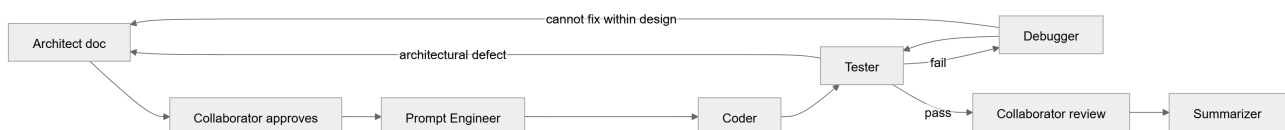


Figure 7 — AI-agent workflow

6.3 Hand-off rules

- Segment granularity.** Each development segment corresponds to one row of the roadmap (Part 3). A segment is small enough to be coded, tested and debugged in one session.
- Single-file discipline.** The Coder always receives the current full `index.html` and returns the full file. Partial diffs are not accepted, because the deliverable is one file and section boundaries are marked by comments (`<!-- ==== TOKENS ==== -->`, `// ==== TILT =====`).
- Requirement traceability.** Every Coder prompt lists the FR/NFR IDs it must satisfy; every Tester report lists the same IDs with pass/fail. Untested IDs are reported as "not covered".
- Constraint guards.** The Tester automatically fails any segment that introduces an external script, stylesheet, or a property other than `transform/opacity` inside a transition or keyframe (NFR-01, NFR-02, NFR-03).
- Debugger boundary.** The Debugger may restructure code inside a module but may not change module boundaries, data model or token names. Such changes escalate to the Architect.

6.4 Escalation to the Architect

Escalation is mandatory when: (a) a requirement cannot be met within the single-file/no-library constraint; (b) 60 fps cannot be achieved with the prescribed CSS-variable motion pattern; (c) a browser support gap contradicts assumption A4; (d) the user changes visual direction materially (e.g.

requests a CMS or multi-page site). Escalations produce a new architecture revision with a decision record.

6.5 Verification tooling available to the team

- VS Code integrated terminal for a local static server (`python -m http.server` or equivalent) and file-size checks.
- Browser DevTools: Performance panel (frame timing), Rendering panel (paint flashing, emulate `prefers-reduced-motion`), Device toolbar (touch emulation, viewport sizes), Accessibility tree.
- Optional lightweight scripted checks (Node script or browser console snippet) that count tiles, verify absence of external resources, and scan for forbidden animated properties. These scripts are test tooling, not part of the deliverable.

7 Technology stack

The stack is minimal by requirement. The value of this section lies in specifying exactly which web platform features are used, why, and what was rejected.

7.1 Selected technologies

Concern	Technology	Justification
Markup	HTML5 with semantic landmarks, native <code><dialog></code> for lightbox	Accessible by default; <code><dialog></code> provides modal semantics, Escape handling and top-layer rendering without library code
Styling	CSS3: custom properties, Grid, <code>clamp()</code> , <code>backdrop-filter</code> , <code>@media (prefers-reduced-motion)</code> , <code>@media (pointer: coarse)</code>	Native theming, fluid type, glass effect and capability queries without JS
Behaviour	Vanilla JavaScript (ES2020), <code>IntersectionObserver</code> , <code>requestAnimationFrame</code> , <code>matchMedia</code>	Required by constraint; all APIs are baseline in evergreen browsers
Typography	Web fonts via <code><link></code> (display serif + grotesque) with system fallbacks and <code>font-display: swap</code>	Only permissible external dependency; degrades gracefully offline
Placeholder imagery	Inline SVG generated from <code>paletteSeed</code> (geometric compositions in accent/neutral tones)	Keeps the file self-contained and looks deliberate rather than stock
Hosting	Any static host (GitHub Pages, Netlify drop, plain web server)	One file; no build
Dev tooling	VS Code, Live Preview or local static server, browser DevTools	No package manager required

7.2 Rejected alternatives

Alternative	Reason rejected
React / Vue / Svelte	Violates "no framework bloat" and "no build step"
Tailwind or any CSS framework	Requires build or large runtime CSS; tokens via custom properties are sufficient
GSAP / Framer Motion / <code>vanilla-tilt.js</code>	External library; spring interpolation is ~30 lines of vanilla JS
Canvas/WebGL for sheen and light	Overkill; CSS radial gradients driven by custom properties achieve the effect on the compositor
External image files	Would break the single-file constraint; inline SVG placeholders are intentional and small
Scroll-driven animations (<code>animation-timeline: scroll()</code>)	Limited browser support in 2024; <code>IntersectionObserver</code> is universal
Custom lightbox <code><div></code> with manual ARIA	<code><dialog></code> gives modal semantics and focus behaviour natively

7.3 Browser support matrix

Feature	Chrome	Firefox	Safari	Edge
CSS custom properties	'	'	'	'
<dialog>	37+	98+	15.4+	79+
backdrop-filter	'	103+	' (prefixed earlier)	'
IntersectionObserver	'	'	12.1+	'
prefers-reduced-motion	'	'	'	'

Where `backdrop-filter` is unavailable, the glass card falls back to a slightly more opaque background via `@supports not (backdrop-filter: blur(1px))`.

7.4 Performance-relevant choices

- `will-change: transform` is applied only while a tile is being interacted with, to avoid excessive compositor layers on the grid.
- Pointer coordinates are read once per frame; tile bounding rectangles are cached on resize and scroll-end, not read per event.
- Fonts are limited to two families, two weights each, subsetting to Latin.

8 Folder/project structure

The runtime deliverable is a single file; however, the workspace keeps a small amount of supporting material for development and testing. Nothing in `tools/` or `docs/` is shipped.

8.1 Workspace layout

```
artist-portfolio/
% % % index.html          # The complete deliverable (HTML + CSS + JS)
% % % README.md          # How to preview, edit content, swap theme, deploy
% % % docs/
%   % % % architecture.md # This document (system-generated PDF source)
% % % tools/
%   % % % check-constraints.mjs # Fails on external scripts/styles, forbidden animated props
%   % % % test-checklist.md    # Manual verification checklist keyed to FR/NFR IDs
```

8.2 Internal structure of index.html

Because all code lives in one file, section markers are mandatory so that Coder prompts and Tester reports can reference exact regions. The order below is fixed.

```
<!-- ===== HEAD: meta, fonts, title ===== -->
<!-- ===== STYLE: TOKENS ===== -->          :root, [data-theme="light"]
<!-- ===== STYLE: BASE & TYPOGRAPHY ===== -->
<!-- ===== STYLE: LAYOUT ===== -->          header, hero, grid, about, contact, footer
<!-- ===== STYLE: CARD / GLASS ===== -->     tile, sheen layer, lift state
<!-- ===== STYLE: LIGHTBOX ===== -->
<!-- ===== STYLE: MOTION & MEDIA QUERIES ===== --> reveal, reduced-motion, coarse pointer,
<!-- ===== BODY: MARKUP ===== -->           semantic skeleton with data-* hooks
<!-- ===== SCRIPT ===== -->
// ===== DATA: ARTWORKS =====
// ===== CAPABILITIES =====
// ===== RENDERER =====
// ===== TILT =====
// ===== POINTER LIGHT =====
// ===== REVEAL =====
// ===== LIGHTBOX =====
// ===== BOOTSTRAP =====
```

8.3 Naming conventions

Element	Convention	Example
CSS custom properties	kebab-case, grouped by prefix	--color-bg, --color-accent, --font-display, --ease-spring, --dur-base
CSS classes	BEM-lite (block, block__element, is-state)	.tile, .tile__sheen, .is-visible, .is-lifted
JS hooks	data-* attributes, never styling classes	data-tilt, data-reveal, data-lightbox-trigger
JS modules	Object literals with init() in const scope, PascalCase	const Tilt = { init(tiles) {...} }
Per-element motion variables	Short, set by JS, consumed by CSS	--rx, --ry, --sx, --sy, --lift

8.4 Content editing surface

The artist edits only two places: the `ARTWORKS` array (titles, media, dimensions, years, alt text, palette seeds) and the copy inside the hero, about and contact markup. Everything else is layout and behaviour and should not need changes when content is replaced. The README documents this explicitly, along with the one-line theme switch (`<html data-theme="light">`).

9 Frontend architecture

The frontend is the application. This section specifies how the three in-file layers (markup, style, behaviour) are organised, how they communicate, and which rules keep the file maintainable despite having no module system. All component names match those introduced in the System architecture section.

9.1 Markup layer

The document skeleton uses semantic landmarks and exposes JavaScript hooks exclusively through `data-*` attributes. Styling classes and behaviour hooks are never shared, so the Coder can restyle without breaking behaviour and vice versa.

```
<header data-nav>...anchor links...</header>
<main>
  <section id="hero" data-reveal>...</section>
  <section id="work" data-reveal>
    <ul class="grid" data-grid></ul>
  </section>
  <section id="about" data-reveal>...</section>
  <section id="contact" data-reveal>...</section>
</main>
<footer>...</footer>
<dialog class="lightbox" data-lightbox>...</dialog>
<div class="pointer-light" data-pointer-light aria-hidden="true"></div>
```

Tiles are generated by **Renderer** from the **ARTWORKS** array. Each tile is a `<li>` containing a `<button data-lightbox-trigger data-tilt>` (so it is natively focusable and activatable by Enter/Space), an inline SVG placeholder, a `.tile__sheen` layer and a caption. A `<noscript>` block renders a static list of artwork captions so NFR-09 (usable without JavaScript) holds even though tiles are JS-rendered; alternatively the Coder may pre-render the tiles in markup and let **Renderer** only enhance them — this is left as a Coder choice with the constraint that both paths must satisfy FR-02.

9.2 Style layer

The style block is organised into six regions in fixed order (tokens !' base !' layout !' card !' lightbox !' motion/ media queries). Two rules govern it:

1. **Token-only colour and motion.** Components reference `--color-*`, `--font-*`, `--space-*`, `--radius-*`, `--dur-*` and `--ease-*` tokens; literal values appear only in the **TOKENS** region.
2. **Static end-states in CSS.** Every animated state (`.is-visible`, `.is-lifted`, tilt transform) is fully described in CSS. JavaScript only toggles a class or writes a custom property.

Token group	Examples	Consumed by
Colour	<code>--color-bg</code> , <code>--color-fg</code> , <code>--color-muted</code> , <code>--color-accent</code> , <code>--color-glass</code>	All components
Typography	<code>--font-display</code> , <code>--font-body</code> , <code>--step--1 ... --step-5</code> (fluid clamp() scale), <code>--tracking-tight</code>	Base, hero, captions
Spacing/ shape	<code>--space-1 ... --space-8</code> , <code>--radius-card</code>	Layout, card
Motion	<code>--dur-base: 400ms</code> , <code>--ease-out: cubic-bezier(.22,1,.36,1)</code> , <code>--tilt-max: 8deg</code>	Card, reveal, lightbox

Token group	Examples	Consumed by
Per-element	--rx, --ry, --sx, --sy, --lift, --lx, --ly	Tile, sheen, pointer light

The tile transform is a single declaration consuming variables, which is the key to keeping motion on the compositor:

```
.tile[data-tilt] {
  transform: perspective(900px)
    rotateX(var(--rx, 0deg))
    rotateY(var(--ry, 0deg))
    translateZ(var(--lift, 0px));
}
.tile__sheen {
  background: radial-gradient(circle at var(--sx,50%) var(--sy,50%),
    rgb(255 255 255 / .18), transparent 60%);
}
```

9.3 Behaviour layer

Each behaviour module is an object literal with an `init()` method and no shared mutable state except through `Capabilities`. Modules never query each other's DOM; they communicate only via CSS custom properties and classes on elements, or via the `ARTWORKS` array. This diagram shows the permitted dependencies.

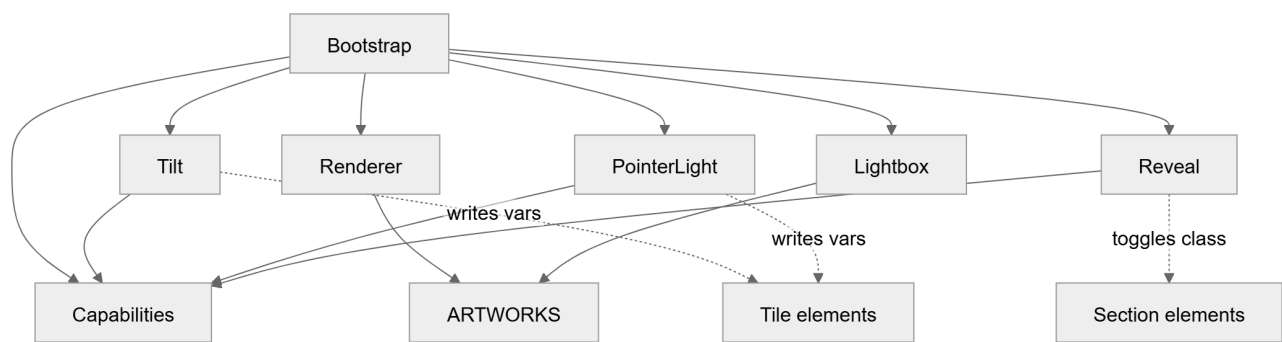


Figure 8 — Frontend architecture

9.4 Responsive strategy

The grid uses `grid-template-columns: repeat(auto-fit, minmax(min(100%, 18rem), 1fr))`, yielding one column at 320 px, two around 640 px and three from roughly 1024 px, with a `max-width` container to keep line lengths editorial on wide screens. Type uses the fluid `--step-*` scale so no breakpoint-specific font sizes are needed. Only two explicit media queries exist: (`pointer: coarse`) to hide the pointer light and flatten tilt, and (`prefers-reduced-motion: reduce`) to zero all durations and reveal content immediately.

10 Backend architecture

The project has **no backend**. This is a deliberate architectural decision derived from the "one self-contained HTML file" constraint and assumption A3 (no form submission endpoint). This section records what would conventionally live on a server and where each responsibility lands instead, so that the absence is an explicit design rather than an omission.

10.1 Responsibility mapping

Conventional backend concern	Where it lives in this project	Notes
Content storage	ARTWORKS array and hero/about/contact markup in index.html	Edited directly by the artist
Templating / rendering	Renderer module (client-side, at load)	Builds tiles from data; ~40 lines
Image serving	Inline SVG generated from paletteSeed	Real images later become <code></code> pointing at a static host or data URI
Contact handling	mailto: link and outbound social links	No spam protection needed because no form exists
Routing	In-page anchor links (<code>#work</code> , <code>#about</code> , <code>#contact</code>)	Single page; no history API usage required
Configuration	TOKENS region and data-theme attribute	No environment variables
Logging / monitoring	Browser DevTools during development only	No runtime telemetry (non-objective)

10.2 Hosting model

The file is served by any static host. The host performs exactly two functions: deliver `index.html` over HTTPS and (optionally) set caching headers. No server-side logic, redirects or headers are required for the site to function; opening the file from disk (`file://`) must also work, which is why the only external references are the optional web-font `<link>` tags with system fallbacks.

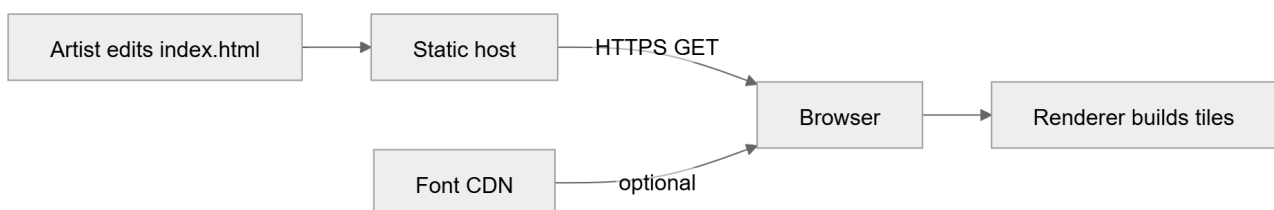


Figure 9 — Backend architecture

10.3 Future extension points

If a backend is later required, the seams are already defined: `ARTWORKS` could be replaced by a `fetch()` of a JSON file with the same record shape (see Data-flow diagrams), and the contact section could point a `<form action>` at a third-party form endpoint. Either change would be a MAJOR architecture revision because it breaks the single-file guarantee (NFR-01), and would require Collaborator approval and a new decision record.

11 Database architecture

There is **no database**. Persistent data consists of a single static array embedded in the script block. This section specifies that array as the project's data schema so that **Renderer**, **Lightbox** and the **Tester** share one contract.

11.1 The ARTWORKS record

Field	Type	Required	Constraints	Used by
id	string	Yes	Unique, kebab-case, used as tile id and lightbox anchor	Renderer, Lightbox
title	string	Yes	1–60 characters; display serif	Renderer, Lightbox
medium	string	Yes	e.g. "Oil on linen"	Renderer caption, Lightbox
dimensions	string	Yes	Free text, e.g. "120 × 90 cm"	Lightbox
year	integer	Yes	Four-digit	Renderer caption, Lightbox
alt	string	Yes	Descriptive, "d 125 characters, never empty"	Renderer (SVG <title>), Lightbox
paletteSeed	string	Yes	Short token selecting one of several SVG compositions/hues	Renderer placeholder generator
src	string	No	Optional real image URL/data URI; when present, replaces the SVG placeholder	Renderer, Lightbox

```
// ==== DATA: ARTWORKS ====
const ARTWORKS = [
  { id: "quiet-field-ii", title: "Quiet Field II", medium: "Oil on linen",
    dimensions: "120 × 90 cm", year: 2023, paletteSeed: "umber-a",
    alt: "Broad horizontal bands of warm umber and bone with a single pale vertical mark."
  }
  ... // 8–11 further records
];
```

11.2 Validation rules

Because there is no schema enforcement at runtime, **Renderer** performs a light validation pass at load: it logs a console warning (development aid, not user-facing) for any record missing a required field and skips that record rather than rendering a broken tile. The **Tester** verifies that exactly 9–12 valid records exist (FR-02) and that every **alt** is non-empty (accessibility baseline).

11.3 Relationships

The data model has a single entity; relationships are between the entity and the UI elements that render it, as previously shown in the Artwork data model diagram. The **id** field is the join key: a tile's **data-id** equals the record **id**, and **Lightbox** resolves the record by **id** rather than by DOM position, so reordering the array reorders the grid without touching the lightbox logic.

11.4 Content lifecycle

1. Placeholder phase: all records use **paletteSeed**, no **src**.

2. Real content phase: the artist adds `src` and updates text fields; layout unchanged.
3. Optional externalisation: the array may be moved to `artworks.json` and fetched — a MAJOR revision (see Backend architecture).

12 API architecture

There is **no network API**. The project exposes no HTTP endpoints and calls none, apart from the optional web-font request made by the browser from `<link>` tags. To keep the behaviour layer testable, this section instead defines the **internal module interfaces** — the contracts between the JavaScript modules — which serve the role an API would in a larger system.

12.1 Internal module contracts

Module	Public method	Input	Effect / output
Capabilities	detect()	none	Returns { reducedMotion, coarsePointer, hasIO } booleans; also sets data-motion="off" on <html> when motion is disallowed
Renderer	render(artworks, gridEl)	ARTWORKS, grid container	Inserts tiles; returns NodeList of tile buttons
Renderer	placeholderSVG(seed, alt)	seed string, alt text	Returns SVG markup string
Tilt	init(tiles, caps)	tile elements, capability flags	Attaches pointermove/pointerleave per tile; starts shared rAF loop on demand
PointerLight	init(lightEl, tiles, caps)	light element, tiles, flags	Attaches one pointermove on window; sets --lx/--ly and --lift on nearby tiles
Reveal	init(sections, caps)	[data-reveal] elements, flags	Observes sections; adds .is-visible once
Lightbox	init(dialogEl, tiles, artworks)	dialog, tiles, data	Wires triggers, keyboard handling, focus trap
Lightbox	open(id), close(), step(± 1)	artwork id / delta	State transitions per the state machine
Bootstrap	start()	none	Executes the sequence shown in the Bootstrap diagram

12.2 Shared motion helper

Tilt and PointerLight both need spring interpolation. A single `spring(current, target, velocity, k, d)` pure function is defined once in the **TILT** region and reused by **PointerLight**. It is the only cross-module shared function besides `Capabilities.detect()`.

```
// Pure; returns [nextValue, nextVelocity]
function spring(cur, target, vel, k = 0.18, d = 0.75) {
  vel = (vel + (target - cur) * k) * d;
  return [cur + vel, vel];
}
```

12.3 Event contracts

Modules communicate with the DOM, not each other. The table lists which native events each module listens to and which DOM outputs it produces, so the Tester can inspect outputs directly.

Module	Listens to	Writes
Tilt	pointermove, pointerleave on tile	--rx, --ry, --sx, --sy; will-change toggled

Module	Listens to	Writes
PointerLight	pointermove on window, resize, scroll (passive, debounced for rect cache)	--lx, --ly on light element; --lift on tiles within radius
Reveal	IntersectionObserver callbacks	.is-visible on sections
Lightbox	click on triggers, keydown (Escape, ArrowLeft, ArrowRight, Tab), click on backdrop, close on dialog	Dialog content, open state, focus

12.4 External requests

Request	Trigger	Failure behaviour
Web fonts (two families)	<link rel="preconnect"> + <link rel="stylesheet"> in HEAD	font-display: swap shows system serif/sans; layout uses size-adjust fallbacks to limit reflow

No other requests are made. The constraint-check tool fails the build if any `<script src>`, `<link rel="stylesheet">` other than the font link, `fetch`, or `XMLHttpRequest` appears in the file.

13 Data-flow diagrams

Data in this project flows from the static **ARTWORKS** array into the DOM at load, and from pointer/scroll input into CSS custom properties at runtime. The two flows are independent: content flow happens once; motion flow runs continuously but writes only compositor-friendly properties.

13.1 Content flow at load

The **Renderer** transforms records into tiles and placeholder SVGs; **Lightbox** later reads the same records by **id**. No data is copied into intermediate structures, so the array remains the single source of truth.

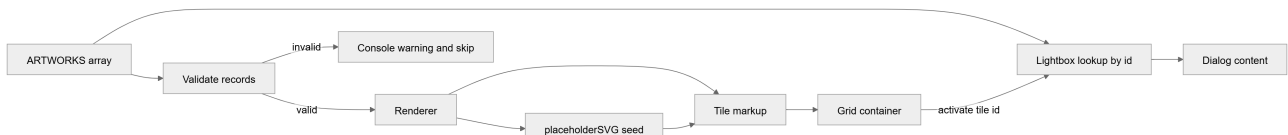


Figure 10 — Data-flow diagrams

13.2 Tilt and sheen flow

Pointer position relative to a tile's cached bounding rectangle becomes target rotation and sheen coordinates. The shared rAF loop springs current values toward targets and writes variables; CSS does the rest. Reading `getBoundingClientRect()` happens only on `pointerenter`, `resize` and debounced `scroll`, never inside `pointermove`.

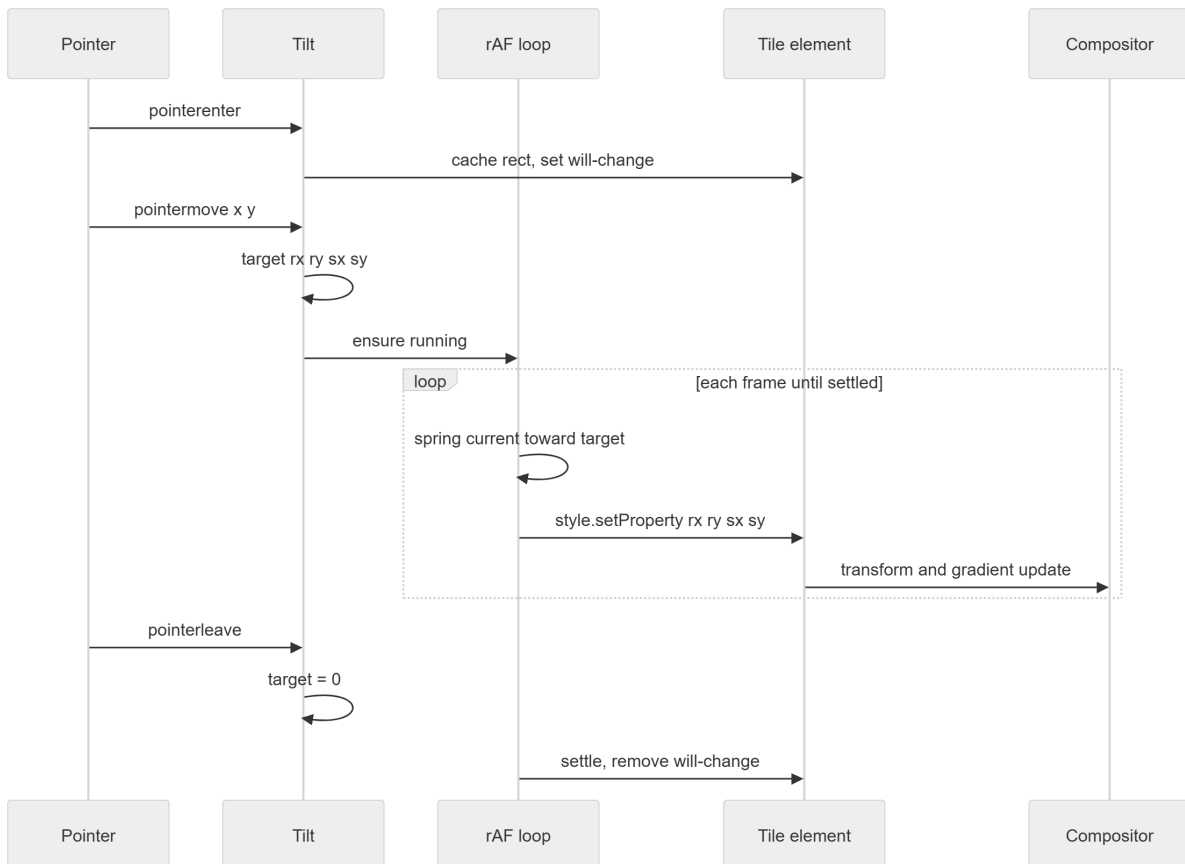


Figure 11 — Data-flow diagrams

13.3 Pointer light and lift flow

A single `pointermove` listener on `window` drives the light element and computes distance to each tile centre from the cached rectangles. Tiles within a radius receive a `--lift` value proportional to proximity; others are reset to 0. The computation is $O(n)$ with $n \leq 12$, executed once per frame.

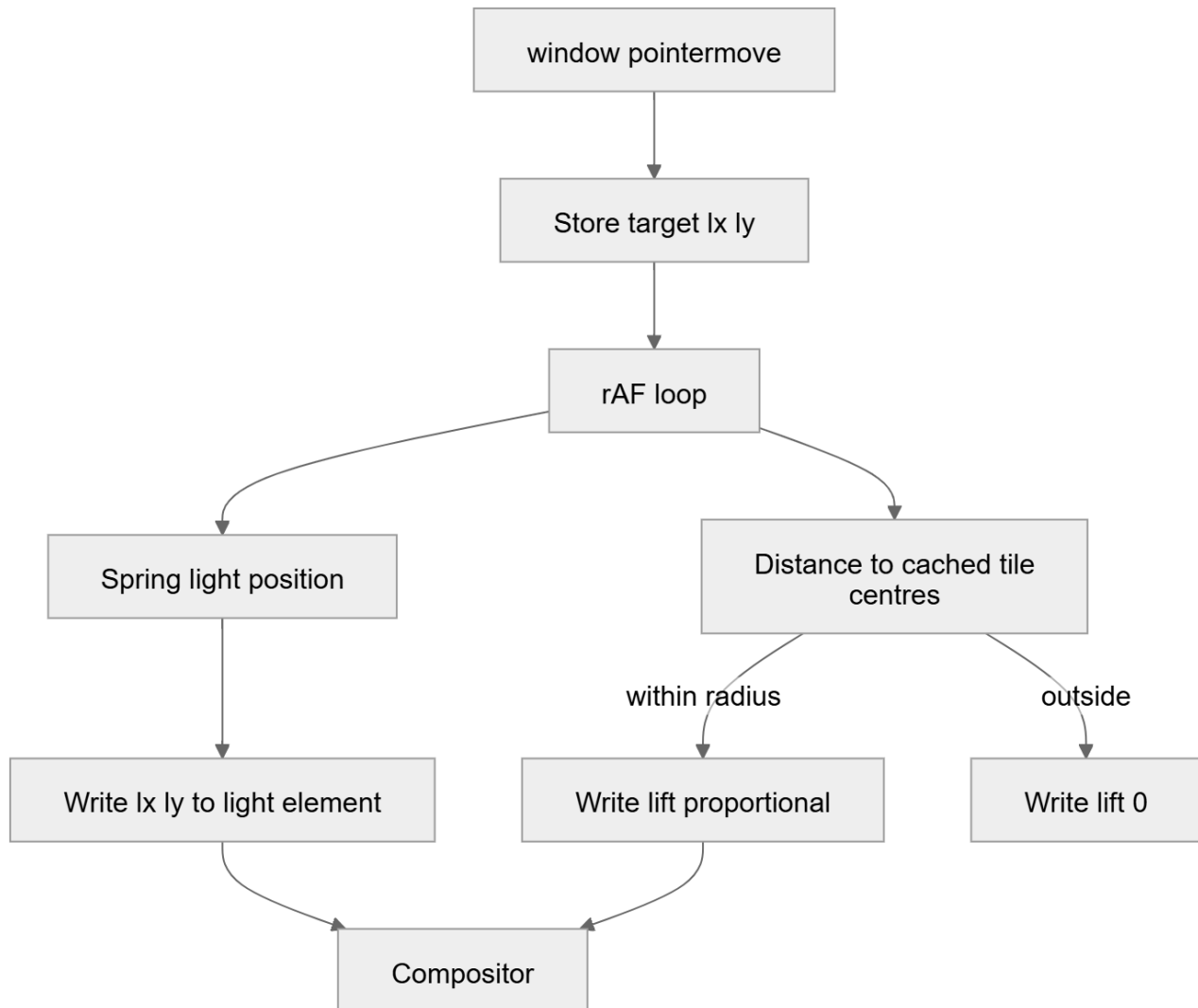


Figure 12 — Data-flow diagrams

13.4 Reveal flow

Sections start with `opacity: 0; transform: translateY(24px)` in CSS (only when JS is active — the class `js` is added to `<html>` by Bootstrap so no-JS visitors see content immediately). The observer adds `.is-visible` and unobserves, so each section animates once.

13.5 Capability gating

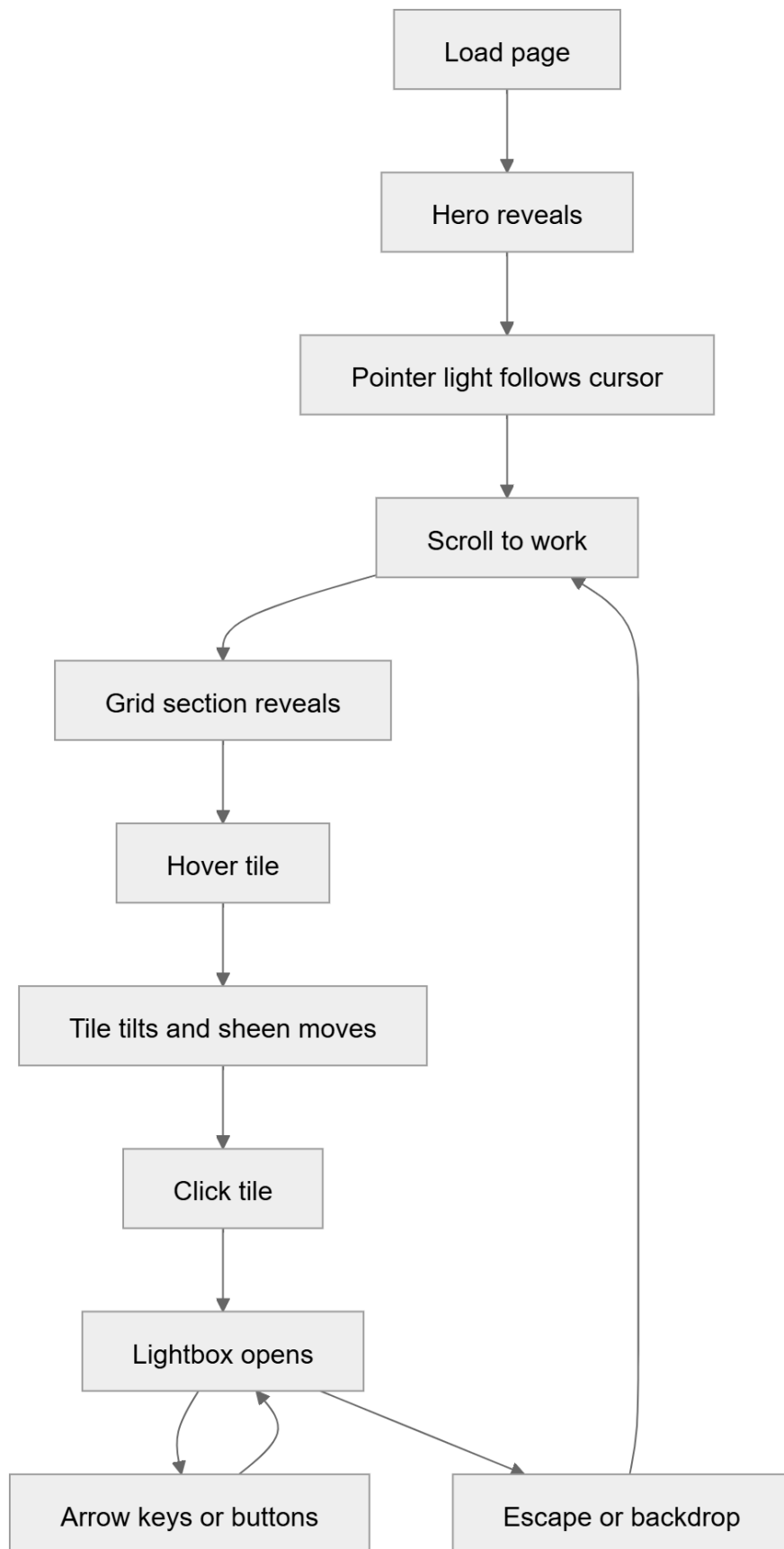
Every runtime flow passes through the `Capabilities` flags at initialisation. When `reducedMotion` or `coarsePointer` is true, `Tilt` and `PointerLight` are never initialised; when `reducedMotion` is true, CSS also sets all `--dur-*` tokens to `0ms` and `Reveal` adds `.is-visible` to all sections synchronously instead of observing.

14 User-flow diagrams

The site has one linear journey with a modal branch. These diagrams complement the page-structure flow already given by focusing on the three input modalities the requirements call out: fine pointer, touch, and keyboard.

14.1 Fine-pointer visitor

A desktop visitor experiences the full motion set. The flow shows where each effect is encountered.

*Figure 13 — User-flow diagrams*

14.2 Touch visitor

On coarse pointers there is no hover, so tilt, sheen and pointer light are not initialised. Cards remain static glass panels; tapping a tile opens the lightbox; swiping inside the lightbox is *not* required (FR-04 specifies prev/next controls, which must be large enough for touch — minimum 44 × 44 px).

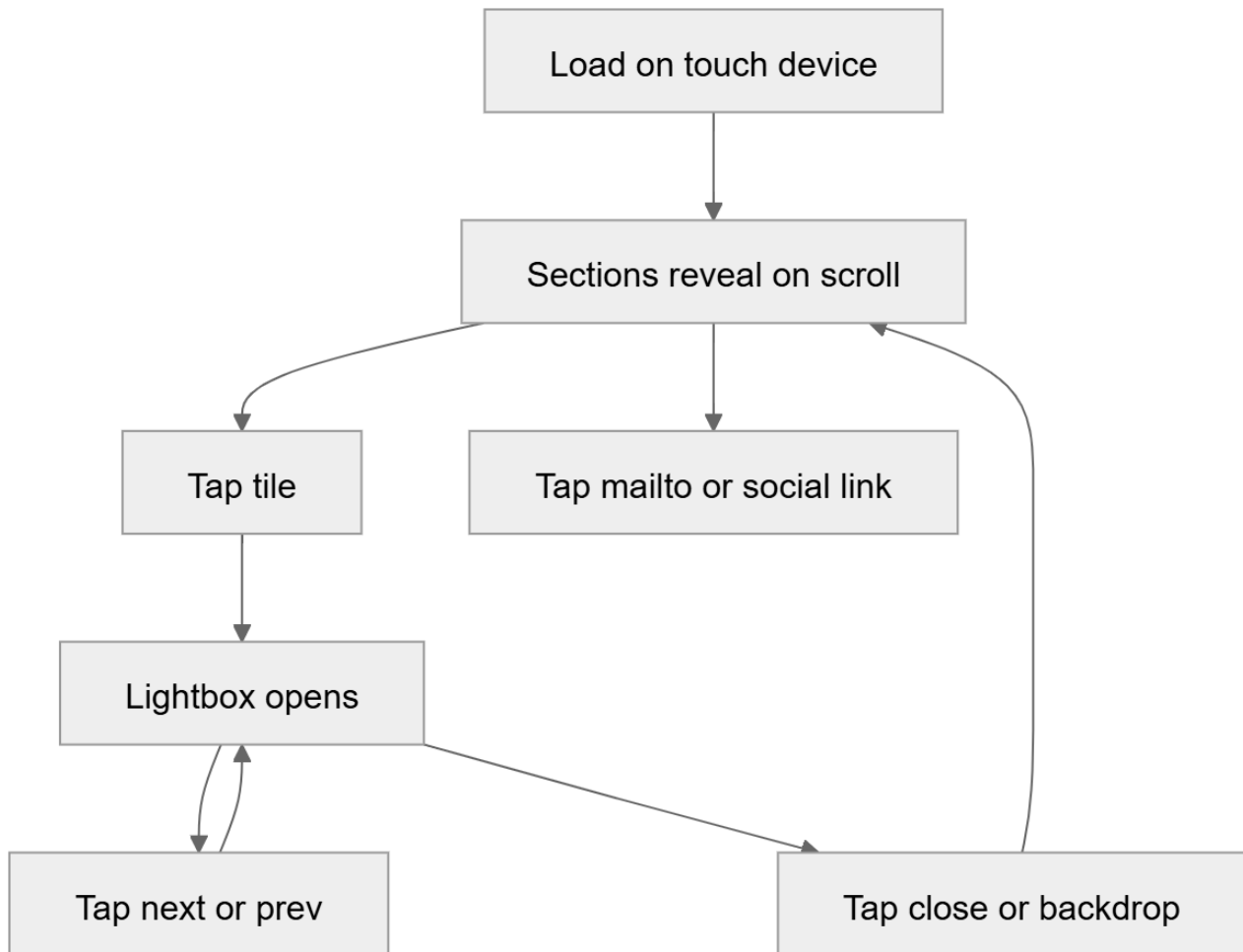


Figure 14 — User-flow diagrams

14.3 Keyboard visitor

Keyboard users must reach every interactive element and never lose focus. The lightbox uses the native `<dialog>` modal behaviour plus an explicit focus trap for browsers where inert-outside behaviour is incomplete.

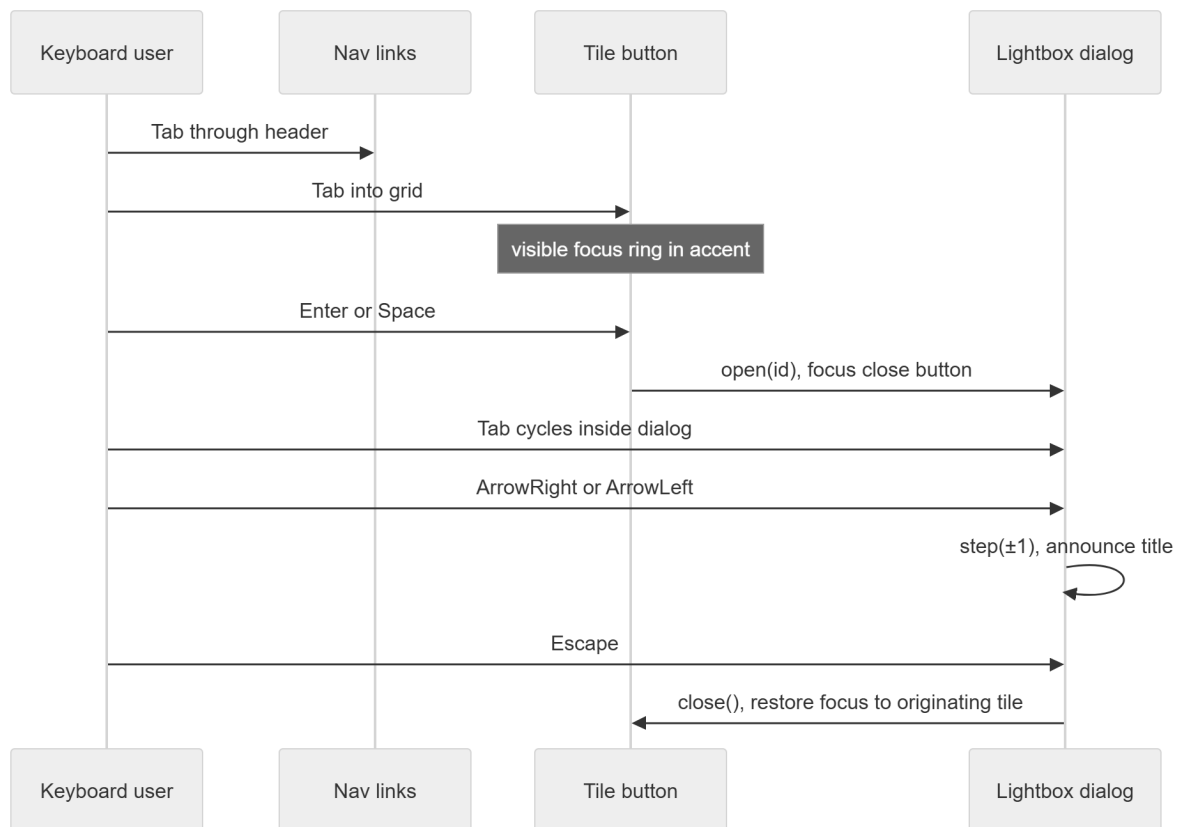


Figure 15 — User-flow diagrams

14.4 Reduced-motion visitor

For `prefers-reduced-motion`: reduce the journey is identical to the touch flow but on any device: all content is visible on load, transitions are instantaneous, and the lightbox opens and closes without fade. This path is a first-class test case (FR-11), not a degraded fallback.

14.5 Focus and announcement rules

Moment	Focus target	Screen-reader consideration
Lightbox opens	Close button	Dialog has aria-labelledby pointing at the artwork title
Next/previous	Stays on the pressed control	Title element has aria-live="polite" so the new title is announced
Lightbox closes	Originating tile button	Prevents focus loss to <body>
Section reveal	No focus change	Reveal is purely visual; content is in the accessibility tree from load

15 AI communication architecture

The product has no AI features, so this section defines how the AI Dev Team roles exchange information while building this specific project. The goal is to make every hand-off traceable to this document and to keep the single `index.html` consistent across roles.

15.1 Message artefacts

From !' To	Artefact	Mandatory contents
Collaborator !' Prompt Engineer	Segment brief	Roadmap segment ID, FR/NFR IDs in scope, approved assumptions, current <code>index.html</code>
Prompt Engineer !' Coder	Coder prompt	Segment ID, exact file regions to edit (e.g. <code>// ==== TILT ====</code>), FR/NFR IDs, forbidden actions (no external files, no non-transform/opacity animation), definition of done
Coder !' Tester	Delivery note + full <code>index.html</code>	List of regions changed, IDs claimed satisfied, any deviations flagged
Tester !' Debugger / Collaborator	Test report	Per-ID pass/fail/not-covered, reproduction steps, DevTools evidence (frame timing, computed styles)
Debugger !' Tester	Patch note + full <code>index.html</code>	Root cause, regions changed, confirmation that module boundaries and token names are unchanged
Any !' Architect	Escalation	Requirement ID, constraint conflicted, proposed options
Collaborator !' Summarizer	Segment closure	Final test report, memory updates (decisions, structure, completed features)

15.2 Communication flow per segment

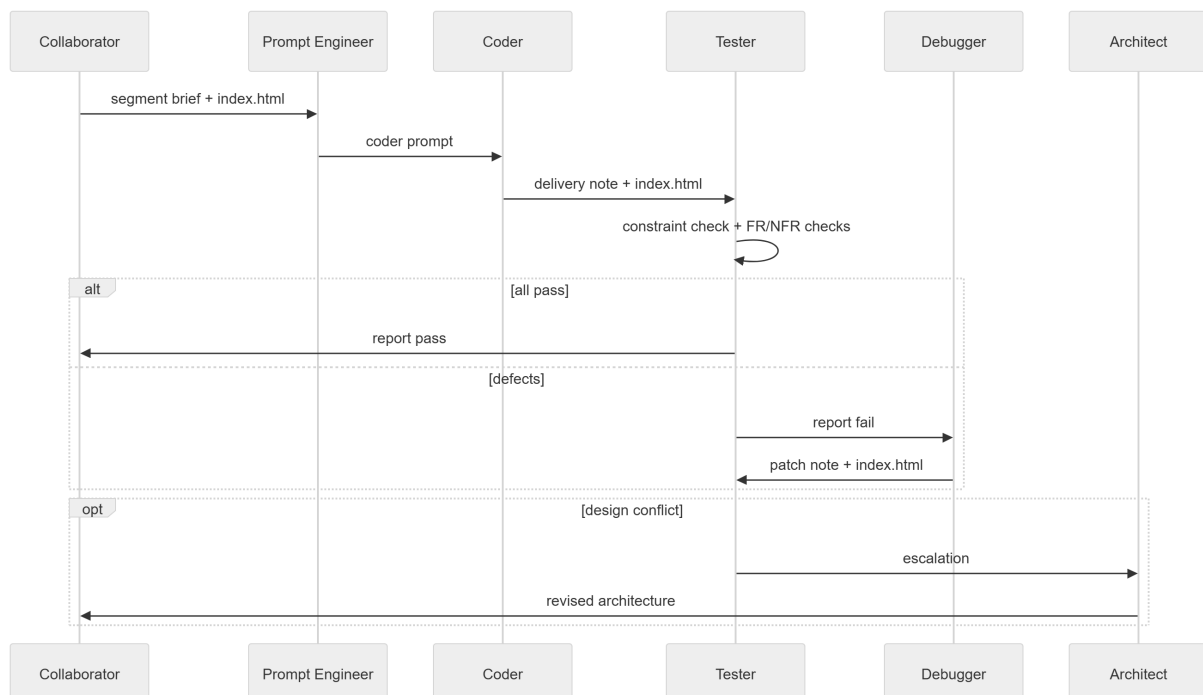


Figure 16 — AI communication architecture

15.3 Shared vocabulary

All roles must use the identifiers defined in this document: module names (`Capabilities`, `Renderer`, `Tilt`, `PointerLight`, `Reveal`, `Lightbox`, `Bootstrap`), region markers, token names, requirement IDs and assumption IDs. A Coder prompt that says "make the cards move" is invalid; one that says "implement `Tilt.init()` in region `// ==== TILT ====` satisfying FR-07, NFR-03, NFR-05" is valid.

15.4 Context the Coder must always receive

Because there is no module system, the Coder cannot see a "file"; it sees the whole application. Every Coder prompt therefore includes: the current complete `index.html`, the token table, the module contract table from the API architecture section, and the constraint list C1–C4. The Prompt Engineer is responsible for trimming other document sections to keep prompts focused.

15.5 Evidence standards for the Tester

- **Performance (NFR-04):** a DevTools Performance recording of 5 s of continuous pointer movement across the grid; pass if no frame exceeds 16.7 ms except the first after `pointerenter` (layer promotion).
- **Property discipline (NFR-03):** output of `tools/check-constraints.mjs` scanning `transition` and `@keyframes` declarations.
- **Reduced motion (FR-11):** screenshot/computed-style evidence with the media feature emulated.
- **Keyboard (NFR-07):** written walkthrough of the sequence in the Keyboard visitor diagram.

16 Security architecture

The attack surface of a static, single-file site with no user input, no backend and no persistence is minimal, but not zero. This section identifies the residual risks and the controls that address them so that the Tester can verify them and future changes do not silently expand the surface.

16.1 Threat assessment

Threat	Relevance	Control
Cross-site scripting (XSS) via content	Low — content is authored by the artist, but Renderer builds HTML from ARTWORKS strings	Renderer uses <code>textContent/createElement</code> for text fields; only the trusted <code>placeholderSVG</code> output is inserted as markup, and it never interpolates user strings except <code>alt</code> , which is escaped into <code><title></code> via <code>textContent</code>
Supply-chain compromise	Low — the only external resource is web fonts	Fonts loaded from a reputable provider over HTTPS with <code>crossorigin</code> ; system fallbacks mean a blocked font host cannot break the page; no third-party scripts at all
Clickjacking / framing	Very low — nothing to hijack	Optional <code>frame-ancestors</code> via host-level CSP if the host supports headers; not required
Mixed content	Low	All URLs (fonts, social links) use <code>https</code> :
Email harvesting from <code>mailto</code> :	Medium for spam	Accept as a trade-off documented to the artist; README suggests an obfuscation option (JS-assembled address) if spam becomes a problem — this would still be vanilla, in-file JS
Reverse tabnabbing via social links	Low	All <code>target="_blank"</code> links carry <code>rel="noopener noreferrer"</code>
Privacy leakage	Low	No analytics, cookies, local storage or fingerprinting; the font request is the only third-party contact and is disclosed in README

16.2 Content Security Policy (CSP)

Because the file must run from `file://` and from arbitrary static hosts, a strict header-based CSP cannot be assumed. A meta-tag CSP is recommended but must allow inline styles and scripts (the whole architecture is inline). The Coder includes:

```
<meta http-equiv="Content-Security-Policy"
      content="default-src 'none'; style-src 'self' 'unsafe-inline' https://
fonts.googleapis.com; font-src https://fonts.gstatic.com; script-src 'unsafe-inline';
      img-src 'self' data:; base-uri 'none'; form-action 'none';">
```

'`unsafe-inline`' is unavoidable here; the value of the policy is `default-src 'none'`, `form-action 'none'` and `base-uri 'none'`, which block any accidental or injected external resource, form post or base hijack. The Tester confirms the page loads without CSP violations in the console.

16.3 Secure-by-construction rules for the Coder

1. Never use `innerHTML` with any string derived from `ARTWORKS` except the output of `placeholderSVG`, which receives only the `seed` (validated against a whitelist of known seeds) and an escaped `alt`.
2. No `eval`, `new Function`, or string-based timers.
3. No inline event handler attributes (`onclick=`); listeners are attached in the script block.
4. All external anchors: `rel="noopener noreferrer"`.

16.4 Verification

Check	Method	Requirement link
No external scripts	tools/check-constraints.mjs	NFR-01, NFR-02
CSP loads clean	Console has zero CSP violations	Security
innerHTML audit	Grep script block; each use justified in delivery note	Security
Link attributes	DOM query for <code>a[target=_blank]</code> lacking <code>rel</code>	Security

17 Authentication architecture

There is **no authentication** and no authorisation in this project. The site is fully public, has no user accounts, no admin interface, no protected content and no personalisation. This section records that decision and its boundaries so that it is a conscious architectural choice rather than an oversight.

17.1 Decision

Aspect	Decision
Visitor identity	Anonymous; no login, no sessions, no cookies
Content editing	Performed by editing index.html directly and redeploying; access control is the responsibility of the hosting/version-control provider (e.g. GitHub repository permissions), outside the deliverable
Lightbox / sections	Publicly accessible; nothing is gated
Contact	Initiated by the visitor through their own mail client via mailto;; no data is received by the site

17.2 Why no authentication is correct here

- The idea specifies a portfolio for public viewing; every requirement (FR-01 ... FR-14) concerns presentation and motion, none concerns restricted access.
- Adding authentication would require a backend or third-party identity service, violating NFR-01 (single file) and C1 (no frameworks/build).
- A client-side "password" in JavaScript would provide no real security and would contradict the honesty of the progressive-enhancement design.

17.3 Trust boundaries

The only trust boundary is between the artist (author of the file) and the hosting platform. The diagram shows where responsibility for access control actually lies.

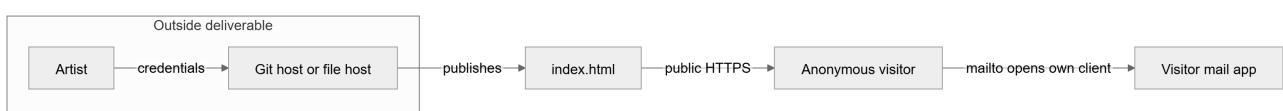


Figure 17 — Authentication architecture

17.4 Conditions that would reintroduce authentication

Any of the following user requests would require a MAJOR architecture revision with a new decision record:

1. A private "works in progress" section visible only to galleries or collectors.
2. An in-browser content editor for the artist.
3. A contact form that stores submissions.
4. Purchase or enquiry workflows tied to specific artworks.

In each case the Architect would evaluate a static-site-plus-service approach (e.g. a hosted form or identity provider) versus a small backend, and the single-file guarantee would be renegotiated with the Collaborator. Until then, the correct authentication architecture for this project is none, and the Tester's only check in this area is that no login UI, cookie or storage access appears in the file.

18 Deployment architecture

Deployment for this project is intentionally trivial: the deliverable is one `index.html`, and "deploying" means copying that file to any HTTPS-capable static host. This section nonetheless specifies the deployment model precisely so that the Tester can verify production behaviour and so that the artist has a repeatable, documented procedure. No build, bundling, minification or transpilation step is part of the pipeline; the file that is tested is byte-for-byte the file that is served.

18.1 Deployment targets

Target	Procedure	Suitability
GitHub Pages	Push <code>index.html</code> to the main branch of a repository with Pages enabled (root or /docs)	Recommended default: free, HTTPS, version history
Netlify Drop	Drag the folder containing <code>index.html</code> onto the Netlify Drop page	Fastest zero-account preview; supports custom domain
Cloudflare Pages / Vercel	Connect repository; framework preset "None", build command empty, output directory /	Good CDN reach; no build configuration required
Plain web server	<code>scp index.html user@host:/var/www/site/</code>	Works with any existing hosting
Local disk (file://)	Double-click <code>index.html</code>	Must work for offline review (NFR-01); fonts fall back to system faces

18.2 Runtime dependencies at deploy time

The only external runtime dependency is the optional web-font request. Everything else — markup, tokens, layout, behaviour, placeholder imagery — is inline. The deployment must therefore satisfy exactly two conditions: serve the file over HTTPS (so that the font CDN is not blocked by mixed-content rules) and serve it with `Content-Type: text/html; charset=utf-8`. No custom headers, redirects, rewrite rules, environment variables or serverless functions exist.



Figure 18 — Deployment architecture

18.3 Release procedure

1. The Tester signs off the segment on the local static server (`python -m http.server 8080` from the workspace root).
2. The Collaborator tags the commit with a semantic version (`v1.0.0`) — the site version is independent of the architecture document version.
3. The file is pushed or uploaded; the host publishes within seconds.
4. A **live smoke test** is run on the published URL: page loads, 9–12 tiles render, lightbox opens and closes via keyboard, `prefers-reduced-motion` emulation removes motion, no console errors, DevTools Network shows only the document and font requests.
5. Rollback is a `git revert` and re-push; because there is one file, rollback is atomic.

18.4 Caching and performance headers

Static hosts apply sensible defaults. Where headers are configurable, the recommended values are `Cache-Control: public, max-age=300` for `index.html` (short, because content edits are frequent) and long-lived caching for fonts (handled by the font CDN). Because NFR-10 targets a file

size under 150 KB, gzip/brotli compression provided by the host is sufficient; no manual minification is performed, so the source remains readable for the artist.

18.5 Content update deployment

When the artist replaces placeholder works with real images, real image files break the single-file constraint by design (lifecycle phase 2 in Database architecture). The recommended pattern is to place images in an `assets/` folder alongside `index.html` on the host and reference them via `src` in `ARTWORKS`. This is a content change, not an architecture change, and does not require a new revision, provided the HTML/CSS/JS remains in one file.

19 Development workflow

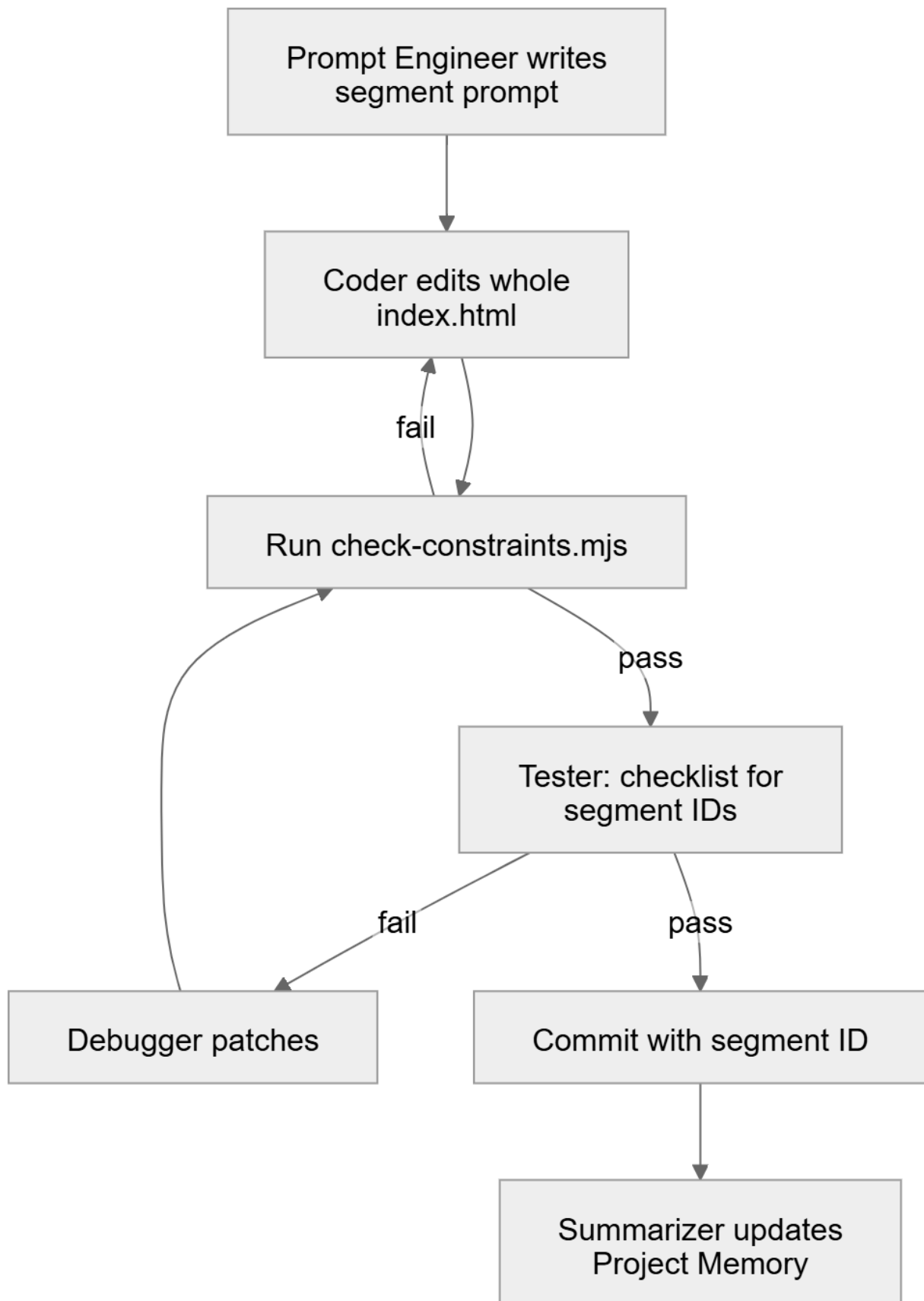
The development workflow is optimised for a single-file codebase edited by a rotating set of AI roles. The primary risk in such a setup is that one role overwrites another's region or introduces a forbidden dependency. The workflow addresses this with fixed region markers, whole-file hand-offs, a mechanical constraint check and requirement-ID traceability, as introduced in the AI-agent workflow section.

19.1 Local environment

Tool	Purpose	Notes
VS Code	Editing index.html, running terminal and tests	No extensions required; Live Preview optional
Local static server	Serve on http://localhost:8080 so <dialog> and fonts behave as in production	python -m http.server 8080 or npx serve (tooling only, not a dependency)
Browser DevTools	Performance, Rendering, Device toolbar, Accessibility tree	Chrome or Firefox; Safari for backdrop-filter verification
Node.js "e 18	Runs tools/check-constraints.mjs	Development-only; the deliverable never depends on Node
Git	Version control, tags, rollback	One commit per development segment

19.2 Segment cycle

Each roadmap segment follows the same cycle. The Prompt Engineer prepares a prompt naming the segment ID, the `index.html` regions to touch, and the FR/NFR IDs to satisfy. The Coder returns the whole file. The constraint check runs before any human or Tester time is spent. The Tester then works through the checklist rows for the segment's IDs.

*Figure 19 — Development workflow*

19.3 Branching and commits

A single `main` branch is sufficient. Every commit message starts with the segment ID and lists the requirement IDs addressed, e.g. `S04 Tilt module: FR-07, NFR-03, NFR-05`. Because the deliverable is one file, merge conflicts are avoided by never working on two segments concurrently; the Collaborator enforces sequential segment execution.

19.4 Coding conventions

The following conventions are binding for the Coder and Debugger and are checked by the Tester during review.

Area	Convention
Regions	Every region opens with its marker comment exactly as listed in Folder/project structure; no code outside a region
JavaScript	ES2020, "use strict", const/let only, no globals except module objects and ARTWORKS; no var, no eval, no inline event handler attributes
Modules	Object literal with <code>init()</code> ; no module reads another module's internal state; communication via CSS variables/classes only
DOM access	Query once at <code>init()</code> , cache references; never <code>querySelector</code> inside a <code>pointermove</code> or <code>rAF</code> callback
Motion	Only transform and opacity in <code>transition/@keyframes</code> ; every duration and easing references a <code>--dur-*/--ease-*</code> token
CSS	Token-only colours; BEM-lite class names; <code>data-*</code> for JS hooks; two media queries plus breakpoints only
Accessibility	Every interactive element is a native <code><button></code> , <code><a></code> or <code><dialog></code> control; <code>aria-*</code> only where native semantics are insufficient
Comments	One-line purpose comment above each module and each non-obvious CSS rule; no commented-out code left in the file
Formatting	Two-space indentation, "d 100-character lines, semicolons, double quotes in HTML, single quotes avoided in JS strings containing HTML

19.5 Definition of done for a segment

A segment is done when: the constraint check passes; every mapped FR/NFR ID is marked pass by the Tester; no console errors or warnings appear on load (except intentional validation warnings when a test record is deliberately invalid); the file opens correctly from `file://` and from the local server; and the Summarizer has recorded the segment.

20 Testing strategy

There is no unit-test framework in the deliverable and none is added, in keeping with the no-dependency constraint. Testing is therefore a combination of **mechanical checks** (a small Node script and browser console snippets), **manual verification** (a checklist keyed to requirement IDs) and **device/emulation testing** in DevTools. Every test is traceable to an FR/NFR ID from the Requirements section, and every segment in the roadmap names the IDs it must pass.

20.1 Test levels

Level	What is tested	How	Automation
Constraint	NFR-01, NFR-02, NFR-03, NFR-10	tools/check-constraints.mjs parses index.html	Fully scripted
Structural	FR-01, FR-02, FR-05, FR-06, FR-13	Browser console snippet counts landmarks, tiles, alt attributes, links	Scripted snippet
Behavioural	FR-03, FR-04, FR-07, FR-08, FR-09, FR-12, FR-14	Manual interaction plus DevTools inspection of computed styles/variables	Manual with recorded steps
Capability	FR-10, FR-11	DevTools emulation of pointer: coarse and prefers-reduced-motion	Manual
Performance	NFR-04, NFR-05	DevTools Performance recording during 5 s of pointer sweeping and scrolling	Manual, threshold-based
Responsive	NFR-06	Device toolbar at 320, 375, 768, 1024, 1440, 2560 px	Manual
Accessibility	NFR-07, NFR-08, NFR-09	Keyboard walkthrough, contrast checker, JS disabled	Manual

20.2 Constraint check specification

tools/check-constraints.mjs reads index.html as text and fails (exit code 1) on any of the following:

- `<script src=` present, or `<link rel="stylesheet"` whose href is not the approved font host.
- Occurrences of `fetch()`, `XMLHttpRequest`, `import()` or `require()`.
- Inside any `transition:` or `@keyframes` block, a property other than `transform`, `opacity`, `background-position` (sheen only) or `visibility` (used with opacity for hidden states).
- Missing region markers or markers out of order.
- File size above 150 KB.
- Fewer than 9 or more than 12 objects in the `ARTWORKS` array, or any record lacking a required field (regex-level check, complemented by the runtime validation).

20.3 Behavioural test cases (excerpt)

ID	Test	Expected result
T-07a	Move pointer to a tile's top-right corner and hold	Computed <code>--rx "H +8deg</code> , <code>--ry "H +8deg</code> within 500 ms; never exceeds 8deg
T-07b	Move pointer off the tile	Variables return to 0 with visible spring settle; will-change removed after settle

ID	Test	Expected result
T-08a	Move pointer within 200 px of a tile without hovering it	--lift > 0 on that tile only; light element's --lx/--ly track pointer
T-09a	Reload, scroll slowly through page	Each section gains .is-visible exactly once; never removed
T-03a	Tab to a tile, press Enter	Dialog opens, focus on close button, background not focusable
T-04a	With dialog open, press ArrowRight three times then Escape	Content advances (wrapping), dialog closes, focus returns to originating tile
T-11a	Emulate prefers-reduced-motion: reduce, reload	No .tile transform variables set, all sections visible immediately, pointer light absent
T-10a	Emulate touch device	Tilt/light listeners not attached (data-motion reflects), tiles static, tap opens lightbox

20.4 Performance test protocol

Record a 6-second Performance profile in Chrome with CPU throttling at 4× while sweeping the pointer across the grid and then scrolling. Pass criteria: no frame longer than 20 ms during the sweep, no "Layout" or "Recalculate Style" entries triggered from within `pointermove` or rAF callbacks (verified via the Bottom-Up view), and the Rendering panel's "Layout Shift Regions" shows nothing after initial load. The Layers panel should show one compositor layer per actively tilted tile and none for idle tiles.

20.5 Test artefacts

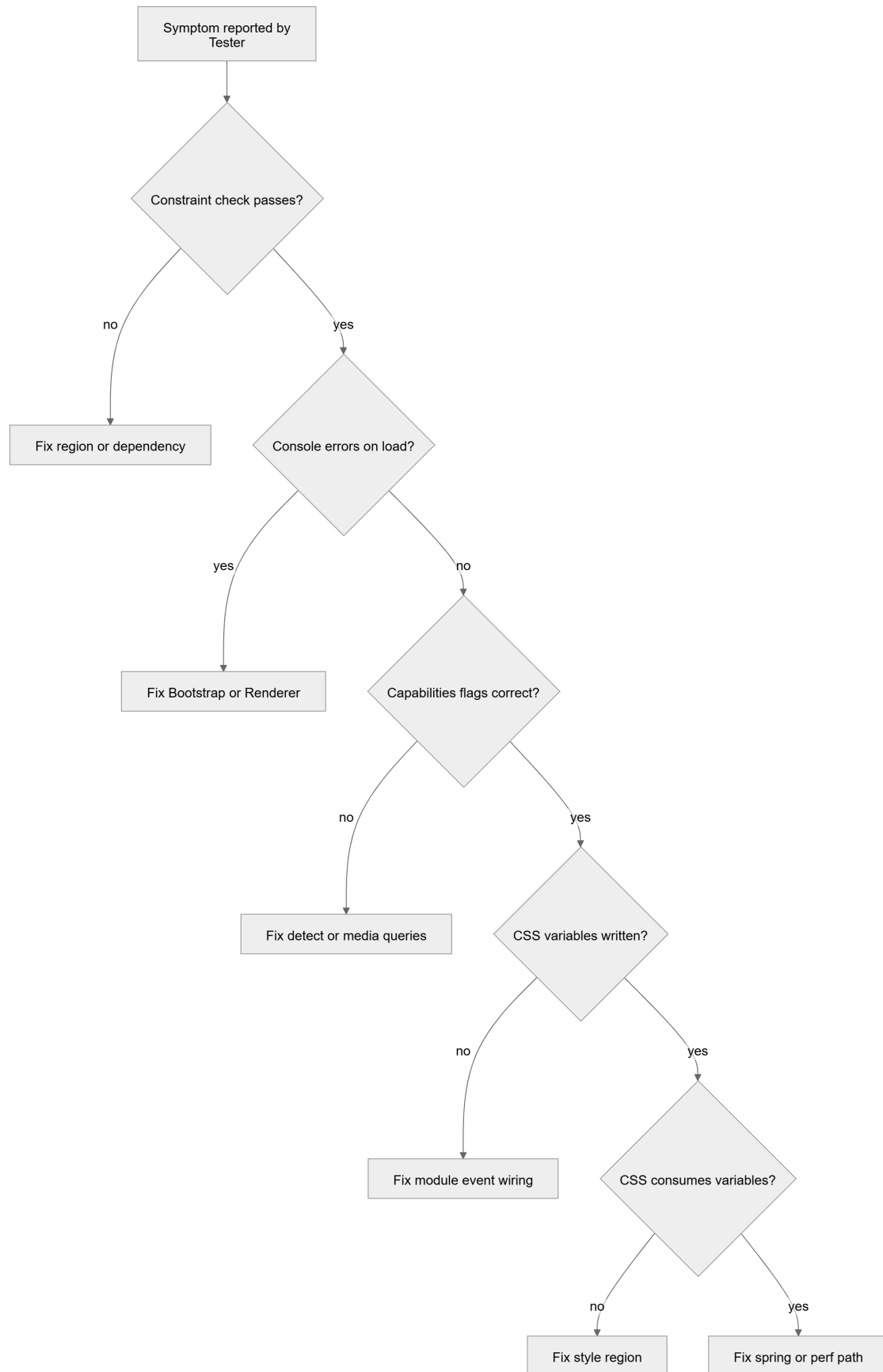
`tools/test-checklist.md` lists every requirement ID with its test steps and a pass/fail column; the Tester copies the relevant rows into each segment report. The checklist is the authoritative test specification and is updated whenever an architecture revision changes a requirement.

21 Debugging strategy

Debugging a single-file, animation-heavy page has two characteristic difficulties: motion bugs are timing-dependent and hard to reproduce from a description, and with no module system a defect in one region can manifest in another. The strategy below gives the Debugger deterministic tools and a fixed diagnostic order, and defines the boundary beyond which a defect becomes an architecture escalation.

21.1 Diagnostic order

The Debugger always works through the layers in this order, stopping at the first layer that explains the symptom.

*Figure 20 — Debugging strategy*

21.2 Built-in debug aids

The deliverable contains a small amount of development-only instrumentation, gated so it has zero cost in normal use:

Aid	Activation	What it shows
Debug attribute	<html data-debug> (added manually in DevTools)	CSS outlines on tiles, visible pointer-light boundary, ::after readout of --rx/--ry on each tile
Capability log	Always, one console.info at bootstrap	{ reducedMotion, coarsePointer, hasIO } so the Tester can confirm emulation took effect
Validation warnings	Always	One console.warn per skipped ARTWORKS record naming the missing field
Motion freeze	window.__motion.freeze() exposed only when data-debug is set	Stops the rAF loop so computed variables can be inspected at rest

These aids are permitted exceptions to the "no globals" convention because they are conditionally created and documented here.

21.3 DevTools techniques by symptom

Symptom	Technique
Jank during pointer movement	Performance panel !' Bottom-Up sorted by self time; look for getBoundingClientRect or style recalculation inside rAF; check rect cache invalidation
Tile tilts wrong direction or overshoots	Freeze motion, read --rx/--ry, compare against pointer offset; inspect spring() constants k/d
Sheen not moving	Elements panel !' Computed !' filter --sx; confirm the .tile__sheen background uses the variables and is not overridden by theme block
Reveal never fires	Check IntersectionObserver thresholds and rootMargin; verify sections have data-reveal and are not display: none
Lightbox focus escapes	Accessibility tree; verify <dialog> opened via showModal() not show()
Reduced motion still animates	Rendering panel emulation active? Check Capabilities.detect() runs before module init (Bootstrap order)
Glass looks flat in Firefox/Safari	Inspect @supports fallback; verify -webkit-backdrop-filter present for older Safari

21.4 Reproduction discipline

Every Tester failure report must include: browser and version, viewport size, emulation settings, the requirement ID, exact steps, and — for motion bugs — a Performance profile export or a short description of pointer path and timing. The Debugger reproduces before patching and records the root cause in one sentence in the commit message. Fixes are minimal and confined to the module where the root cause lies; a fix touching more than one module is a signal to re-check the module boundary rather than to widen the patch.

21.5 Escalation boundary

The Debugger escalates to the Architect when the fix would require a third media query, a new shared function beyond `spring()` and `Capabilities.detect()`, an additional external resource, animation

of a property outside the allowed set, or a change to the [ARTWORKS](#) record shape. These are architectural changes and produce a decision record in a new document revision.

22 Error-handling strategy

Because the page has no network calls, no user input beyond pointer/keyboard, and no persistent state, the error surface is small. The strategy is **fail silent, degrade to static**: any behaviour module that encounters an unexpected condition disables itself and leaves the fully styled static page intact. The user never sees an error message, because every error path ends in a state that is already a valid, designed presentation (the same state shown under `prefers-reduced-motion`).

22.1 Error categories and handling

Category	Example	Handling	User-visible effect
Missing platform API	IntersectionObserver undefined in an old browser	Capabilities.hasIO = false; Reveal skipped; CSS rule <code>html:not([data-io]) [data-reveal] { opacity: 1; transform: none }</code> shows sections	Content visible, no reveal animation
Invalid data	ARTWORKS record missing alt	Renderer logs <code>console.warn</code> , skips record	One fewer tile; Tester catches via count
Empty or short data	Fewer than 9 valid records	Rendered as-is; grid still valid; warning logged	Fewer tiles; flagged in test
Missing DOM hook	<code>[data-grid]</code> absent because markup was edited	Bootstrap checks each required element and skips dependent module with a <code>console.warn</code>	Static page without that behaviour
Module runtime exception	Unexpected null inside <code>Tilt.init</code>	Bootstrap wraps each <code>init()</code> in <code>try/catch</code> ; failure disables only that module	Other modules continue
Font load failure	CDN unreachable, offline <code>file://</code>	<code>font-display</code> : swap with size-adjusted system fallbacks	Slight typographic difference, no layout break
<code>backdrop-filter</code> unsupported	Firefox < 103	<code>@supports</code> not fallback increases --color-glass opacity	Slightly less translucent cards
Pointer left window mid-animation	<code>pointerleave</code> not fired on some platforms	<code>rAF</code> loop targets reset on window blur and <code>pointerout</code> with <code>relatedTarget === null</code>	Tiles settle to rest
<code><dialog></code> unsupported	Very old Safari	Lightbox checks <code>typeof dialog.showModal === "function"</code> ; otherwise tile buttons become anchors to <code>#id</code> captions	No lightbox; captions still readable

22.2 Bootstrap error containment

The `Bootstrap.start()` sequence is the single place where errors are caught. Each module initialisation is isolated so a fault in a later module cannot prevent an earlier one from working, and vice versa.

```
// ==== BOOTSTRAP ==== (illustrative pattern, not full implementation)
const steps = [
  ["Lightbox", () => Lightbox.init(dialogEl, tiles, ARTWORKS)],
  ["Reveal", () => caps.hasIO && Reveal.init(sections, caps)],
  ["Tilt", () => caps.motionOk && Tilt.init(tiles, caps)],
];
for (const [name, run] of steps) {
```

```
try { run(); } catch (err) { console.warn(`[portfolio] ${name} disabled:`, err); }
```

22.3 Guarantees

1. **No unhandled exceptions** reach the console during load or interaction in supported browsers (verified by Tester: console clear except intentional warnings).
2. **No error UI.** There are no toasts, alerts or error banners; the static design is the fallback for every failure.
3. **No state corruption.** The only stateful component, `Lightbox`, resets to `Closed` on any thrown error inside its handlers, and always restores focus to the triggering tile if one is recorded.
4. **Idempotent initialisation.** Calling `Bootstrap.start()` twice (e.g. via a debug console) does not double-attach listeners; modules set a `data-ready` attribute on their root element and return early if present.

22.4 What is deliberately not handled

Runtime telemetry, error reporting services and user-facing diagnostics are non-objectives. Errors are observable only in the developer console; this is acceptable because the page has no server state to protect and the fallback is always the designed static page.

23 Future scalability

"Scalability" for a single-page portfolio is about content growth and feature growth rather than traffic; a static file on a CDN scales to any realistic audience without change. This section identifies where the architecture will strain as the artist's needs grow and pre-defines the seams so that future work can be scoped as MINOR or MAJOR revisions with predictable consequences.

23.1 Growth dimensions

Dimension	Current design	Comfortable limit	Beyond the limit
Number of works	9–12 inline SVG tiles, all rendered at load	~30 tiles with real images and <code>loading="lazy"</code>	Pagination or series filtering; <code>IntersectionObserver</code> already present for lazy rendering
Image weight	Inline SVG placeholders ("H1 KB each")	Real images as <code>src</code> on the same host, "d 300 KB each, <code>srcset</code> for 2×	Image CDN or <code><picture></code> with AVIF/ WebP; still a content change
Content editing	Direct HTML/JS edits	One artist, occasional updates	Move ARTWORKS to <code>artworks.json</code> + <code>fetch()</code> (MAJOR: breaks NFR-01) or a headless CMS
Sections	Four fixed sections	Adding "Exhibitions" or "Press" as further data-reveal sections	Multi-page site requires routing — out of scope, new architecture
Themes	Dark default, light override	Two or three token sets	Runtime theme toggle is a MINOR addition (one button, <code>localStorage</code>)
Motion effects	Tilt, sheen, pointer light, reveal	Adding parallax hero or magnetic buttons using the same rAF/CSS-variable pattern	Effects requiring layout reads per frame are architecturally forbidden
Contact	<code>mailto:</code> and links	—	Third-party form endpoint (Formspree-style) as a <code><form action></code> ; MINOR if no JS added

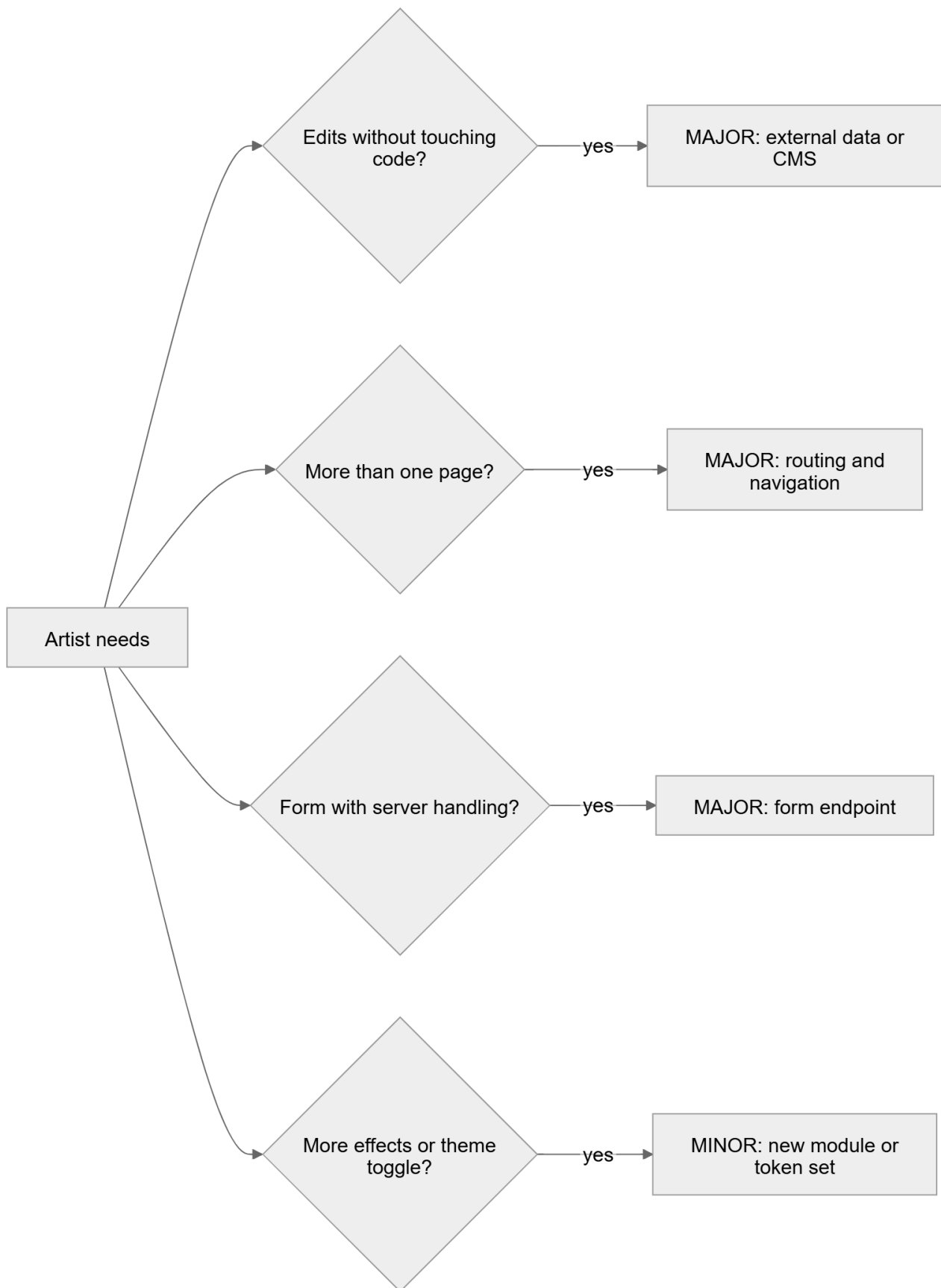
23.2 Pre-defined seams

- **Data seam.** Everything downstream of `ARTWORKS` accepts any array of records with the documented shape. Replacing the literal with a fetched array changes only the first lines of the `DATA` region and the `Bootstrap` start (await before render).
- **Image seam.** `Renderer` already prefers `src` over `placeholderSVG`. Adding `srcset/sizes` fields to the record is a backward-compatible extension.
- **Effect seam.** New motion modules follow the `init(elements, caps)` contract, reuse `spring()`, and are appended to the `Bootstrap` steps list; no existing module changes.
- **Section seam.** New sections need only `data-reveal` and an anchor in `[data-nav]`.

23.3 Performance ceiling

The shared rAF loop iterates over active tiles only; with the proximity radius of the pointer light, at most 3–6 tiles are active at once regardless of grid size, so the motion cost is bounded independently of the number of works. The reveal observer has one entry per section, not per tile. The theoretical ceiling is compositor memory: each actively tilted tile promotes to its own layer, which is why `will-change` is applied only during interaction.

23.4 Signals that a MAJOR revision is due

*Figure 21 — Future scalability*

24 Development phases

Development is organised into four phases that group the roadmap segments by risk and dependency. Phase 1 establishes the static, fully accessible page — the fallback state for everything else. Phase 2 adds the behaviour that has the highest technical risk (60 fps motion). Phase 3 completes interactive features and capability handling. Phase 4 hardens and ships. Each phase ends with a Collaborator checkpoint.

24.1 Phase overview

Phase	Name	Goal	Segments	Exit criterion
P1	Static foundation	Complete, styled, accessible page with no JavaScript motion	S01–S04	Page passes structural, responsive and accessibility tests with JS disabled; constraint check passes
P2	Core motion	Tilt, sheen and pointer light at 60 fps	S05–S06	Performance protocol passes; motion disabled correctly under emulation
P3	Interaction and reveal	Lightbox, scroll reveal, theme switch, navigation	S07–S09	All FR-03/04/09/12/14 tests pass; keyboard walkthrough clean
P4	Hardening and release	Error containment, placeholder polish, docs, deployment	S10–S12	Live smoke test passes; README and checklist complete

24.2 Relative effort

The chart shows estimated relative effort per phase (in Coder/Tester sessions). Motion carries the most risk and the most iteration with the Debugger.

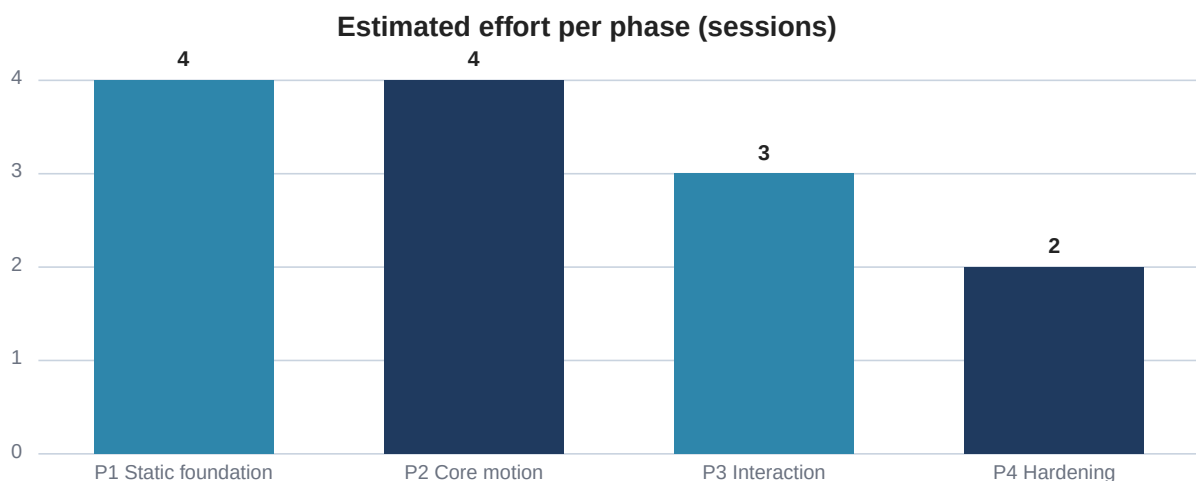


Figure 22 — Estimated effort per phase (sessions)

24.3 Phase dependencies and checkpoints

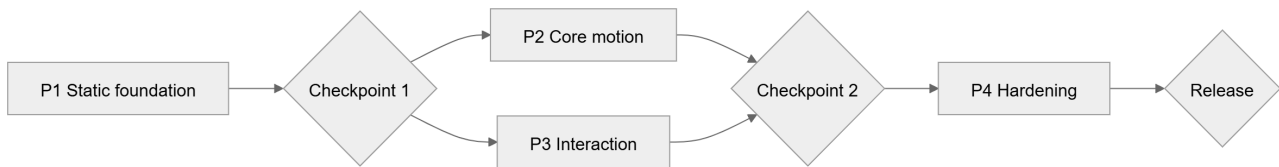


Figure 23 — Development phases

Phases P2 and P3 are independent of each other once P1 is complete and may be sequenced in either order by the Collaborator; the default order puts motion first because it carries the escalation risk (60 fps not achievable !' Architect). Running P2 early surfaces that risk before effort is spent on P3 features.

24.4 Checkpoint contents

- **Checkpoint 1** confirms assumptions A1–A6 against the rendered static page (colours, type, placeholder style) — this is the moment the user sees the visual direction for the first time and can redirect cheaply.
- **Checkpoint 2** reviews the full interactive experience on desktop and in touch emulation; the Collaborator decides whether any "Could" requirements (FR-14, NFR-10) are dropped.
- **Release** is the live smoke test and tag described in Deployment architecture.

25 Implementation roadmap

The roadmap decomposes the phases into twelve segments, each completable by the Coder in one focused task and independently testable by the Tester against named requirement IDs. Segments are ordered by dependency; a segment may begin only when its predecessors have passed. Each row names the `index.html` regions the Coder may modify, which the Prompt Engineer copies into the prompt.

ID	Phase	Segment	Regions touched	Requirement IDs	Depend s on
S01	P1	Project scaffolding	All region markers, HEAD, BODY skeleton, tools/, README stub	NFR-01, NFR-02	—
S02	P1	Design tokens and typography	STYLE: TOKENS, STYLE: BASE & TYPOGRAPHY	O2, NFR-08, NFR-11	S01
S03	P1	Layout, hero, about, contact	STYLE: LAYOUT, BODY: MARKUP, STYLE: MOTION & MEDIA QUERIES (breakpoints only)	FR-01, FR-05, FR-06, NFR-06, NFR-09	S02
S04	P1	Data, Renderer and glass tiles	DATA: ARTWORKS, RENDERER, STYLE: CARD / GLASS, BOOTSTRAP (skeleton), noscript	FR-02, FR-13, O9	S03
S05	P2	Capabilities and Tilt with sheen	CAPABILITIES, TILT, STYLE: CARD / GLASS (variables), MEDIA QUERIES (coarse, reduced)	FR-07, FR-10, FR-11, NFR-03, NFR-05	S04
S06	P2	PointerLight and proximity lift	POINTER LIGHT, BODY (light element), STYLE: CARD / GLASS (lift)	FR-08, NFR-04	S05
S07	P3	Lightbox	LIGHTBOX, STYLE: LIGHTBOX, BODY (dialog)	FR-03, FR-04, NFR-07	S04
S08	P3	Scroll reveal	REVEAL, STYLE: MOTION & MEDIA QUERIES	FR-09, FR-11	S05
S09	P3	Theme switch and navigation	STYLE: TOKENS (light), BODY (header), STYLE: LAYOUT	FR-12, FR-14	S03
S10	P4	Error containment and debug aids	BOOTSTRAP, all module init guards, STYLE (fallbacks)	Error-handling guarantees, NFR-09	S05–S09
S11	P4	Performance and constraint hardening	Any region (optimisation only), tools/check-constraints.mjs	NFR-04, NFR-05, NFR-10	S10
S12	P4	Documentation and deployment	README.md, tools/test-checklist.md, live deploy	Deployment smoke test	S11

25.1 Segment details

- **S01 Project scaffolding.** Creates `index.html` with every region marker in the fixed order, empty `<style>` and `<script>` blocks, the semantic landmark skeleton with `data-*` hooks, font `<link>` tags, and `tools/check-constraints.mjs` with the marker-order and external-resource rules. Deliverable: a valid, empty page that passes the constraint check.
- **S02 Design tokens and typography.** Implements the dark token set, fluid type scale (`--step--1...--step-5`), letter-spacing tokens, motion tokens (`--dur-base`, `--ease-out`, `--tilt-max`), and base element styles. Tester verifies contrast and font fallback behaviour.

- **S03 Layout, hero, about, contact.** Container, grid CSS, section spacing, hero composition, about with portrait placeholder, contact links, footer, responsive breakpoints. Tested at six viewport widths and with JS disabled.
- **S04 Data, Renderer and glass tiles.** Twelve realistic `ARTWORKS` records, `placeholderSVG()` with at least four compositions × hue seeds, tile markup with `<button>`, sheen layer (static), glass treatment with `@supports` fallback, validation warnings, `<noscript>` captions, minimal `Bootstrap` that renders.
- **S05 Capabilities and Tilt.** `detect()`, `data-motion` attribute, `spring()`, per-tile pointer tracking, rAF loop with self-suspend, `will-change` toggling, sheen variables; coarse-pointer and reduced-motion media queries.
- **S06 PointerLight.** Fixed radial light element, window `pointermove`, cached tile rects with debounced invalidation, proximity -- `lift`, visibility rules under coarse pointer.
- **S07 Lightbox.** `<dialog>` with `showModal()`, content population by `id`, prev/next wrap, Escape/arrow/backdrop handling, focus restore, `showModal` capability fallback.
- **S08 Scroll reveal.** Observer with sensible `rootMargin`, once-only `.is-visible`, CSS fade-and-rise using tokens, immediate visibility under reduced motion or without IO.
- **S09 Theme switch and navigation.** `[data-theme="light"]` token overrides tuned for bone-white glass, minimal sticky header with anchor links and current-section styling that uses only opacity.
- **S10 Error containment.** `try/catch` per module, DOM-hook checks, idempotent `init`, `data-debug` aids, `pointerout/blur` settle behaviour.
- **S11 Performance hardening.** Performance protocol run, layer audit, file-size check, extension of `check-constraints.mjs` to forbidden-property scanning.
- **S12 Documentation and deployment.** README (preview, edit content, swap theme, deploy), completed test checklist, tag and publish, live smoke test.

25.2 Requirement coverage

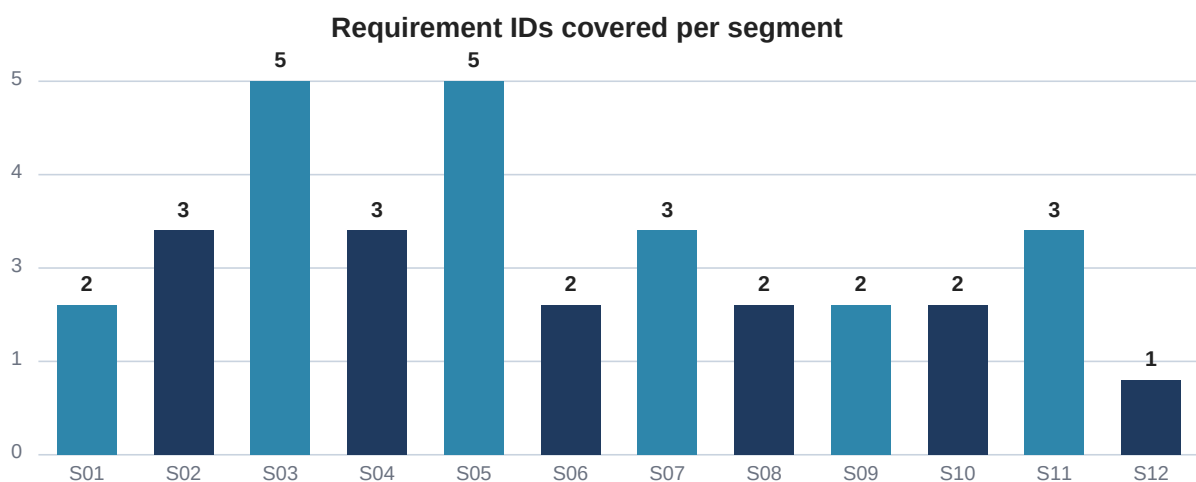


Figure 24 — Requirement IDs covered per segment

26 Architecture decisions

This section records the significant decisions made in version 1.0. Each record states the context, the decision, the alternatives considered and the consequences, so that later revisions can revisit them with full knowledge of the trade-offs. Decisions are numbered ADR-1 ... ADR-10 (ADR: Architecture Decision Record).

26.1 ADR-1: Treat the original idea as the requirements baseline

- **Context:** Upstream Requirements Summary, Refined Requirements and Research Report were empty; the Summarizer reported it could not proceed. The original idea in Project Memory is detailed and internally consistent.
- **Decision:** Derive requirements directly from the original idea, enumerate assumptions A1–A6, and require Collaborator confirmation at Checkpoint 1.
- **Alternatives:** Halt and return to the user (delays with little information gain); invent a requirements summary (violates "never invent facts").
- **Consequences:** Fast start; risk that a confirmed assumption is wrong is mitigated by an early visual checkpoint after P1.

26.2 ADR-2: Single self-contained HTML file with region markers

- **Context:** User mandates one file, no build step, no framework.
- **Decision:** One `index.html` containing all HTML, CSS and JS, organised into fixed, marker-delimited regions; whole-file hand-offs between roles.
- **Alternatives:** Separate `style.css/app.js` (cleaner but violates the constraint); ES modules with `type="module"` inline (no import benefit without files).
- **Consequences:** Trivial deployment; conflict risk managed by sequential segments and markers; the constraint check enforces the boundary mechanically.

26.3 ADR-3: CSS-custom-property-driven motion on the compositor

- **Context:** 60 fps and "transforms and opacity only" requirements; tilt, sheen and light must react to pointer position.
- **Decision:** JS writes per-element variables (`--rx`, `--ry`, `--sx`, `--sy`, `--lift`, `--lx`, `--ly`) from a single rAF loop; CSS consumes them in `transform` and gradient positions.
- **Alternatives:** Directly setting `style.transform` strings per element (equivalent performance, but couples JS to CSS structure); Web Animations API (awkward for continuous pointer-driven targets); Canvas/WebGL (over-engineered).
- **Consequences:** Clean separation, single DOM-write site, easy inspection in DevTools; requires discipline to keep all end-states in CSS.

26.4 ADR-4: Spring interpolation in vanilla JS rather than a library

- **Context:** "Spring-damped rather than linear" easing required; no libraries permitted.
- **Decision:** A ~5-line pure `spring()` function shared by Tilt and PointerLight, with tunable `k` and `d` constants defaulting to a "H400 ms settle".
- **Alternatives:** CSS `transition` on the transform (linear-ish, cannot track a moving target smoothly); GSAP/popmotion (forbidden).
- **Consequences:** Full control and zero dependencies; tuning is empirical and documented in the

TOKENS/TILT regions.

26.5 ADR-5: Native `<dialog>` for the lightbox

- **Context:** Lightbox must trap focus, close on Escape, and be keyboard accessible with minimal code.
- **Decision:** Use `<dialog>` with `showModal()`; add only arrow-key navigation and backdrop click.
- **Alternatives:** Custom `<div role="dialog">` with manual focus trap and `inert` handling (more code, more bugs).
- **Consequences:** Excellent accessibility with little code; requires Safari 15.4+ (accepted under A4) with an anchor-based fallback.

26.6 ADR-6: Inline SVG placeholders generated from a palette seed

- **Context:** Placeholders must look intentional and the file must be self-contained.
- **Decision:** `Renderer.placeholderSVG(seed, alt)` produces geometric compositions in accent/neutral tones; `src` overrides when real images arrive.
- **Alternatives:** External placeholder service (network dependency); CSS gradients only (risks the "2013" look); data-URI raster images (large).
- **Consequences:** Tiny, deliberate, themeable placeholders; a smooth path to real images via the `src` field.

26.7 ADR-7: Progressive enhancement with capability gating

- **Context:** Reduced-motion, touch and old-browser requirements; page must be usable without JS.
- **Decision:** `Capabilities.detect()` runs first; each module self-disables; CSS provides all static end-states; `<noscript>` captions.
- **Alternatives:** Always-on JS with runtime checks scattered through modules (harder to test).
- **Consequences:** One place to emulate/verify capability behaviour; the designed static page is the universal fallback.

26.8 ADR-8: No backend, mailto: contact

- **Context:** Single file, no server components in the idea.
- **Decision:** No server, no form; contact via `mailto:` and social links.
- **Alternatives:** Third-party form endpoint (adds network dependency and spam handling).
- **Consequences:** Zero infrastructure; a form can be added later as a MINOR/MAJOR change per Future scalability.

26.9 ADR-9: Manual, checklist-driven testing plus a mechanical constraint check

- **Context:** No test framework is permitted in the deliverable; motion behaviour is visual.
- **Decision:** `tools/check-constraints.mjs` for mechanical rules; `tools/test-checklist.md` keyed to FR/NFR IDs for behaviour; DevTools protocols for performance.
- **Alternatives:** Playwright suite (heavy dependency for a one-file project, though permissible as tooling; deferred).
- **Consequences:** Low overhead, full traceability; relies on Tester discipline; Playwright remains an option in a MINOR revision.

26.10 ADR-10: Motion phase before interaction phase

- **Context:** 60 fps is the requirement most likely to force an architecture escalation.
- **Decision:** Schedule P2 (motion) immediately after the static foundation.
- **Alternatives:** Feature-complete first, optimise later.
- **Consequences:** Risk surfaces early; the Collaborator may reorder P2/P3 if the user wants to see the lightbox first.

27 Assumptions and limitations

This final section consolidates the assumptions on which the architecture rests and the limitations the team and the user must accept. Assumptions A1–A6 were introduced in the Project overview and are restated here with their verification point; further assumptions discovered while designing Parts 2 and 3 are added as A7–A10.

27.1 Assumptions

#	Assumption	Verified at	Owner
A1	The original idea in Project Memory is the authoritative requirements source	Collaborator approval of this document	Collaborator
A2	Placeholder images may be generated in-file as inline SVG	Checkpoint 1 (visual review of S04)	User via Collaborator
A3	No backend; contact via mailto: and social links	Approval of this document	Collaborator
A4	Target browsers are the last two versions of Chrome, Firefox, Safari and Edge	Approval; Tester uses these browsers	Collaborator
A5	Hosting is any static host; GitHub Pages is the default	S12	Collaborator
A6	Real artwork content will be swapped in by editing ARTWORKS and a few markup blocks	README review at S12	User
A7	Two web-font families from a public CDN are acceptable as the sole external dependency	S02	Collaborator
A8	Dark (near-black) is the default theme; bone white is the alternate	Checkpoint 1	User via Collaborator
A9	Node.js is available on the development machine for the constraint check (tooling only)	S01	Prompt Engineer / Coder
A10	"Mid-range laptop" for the 60 fps test means Chrome with 4× CPU throttling on the Tester's machine	S05/S11	Tester

27.2 Limitations

- **Content management.** The artist must edit the HTML file directly; there is no admin UI. This is a consequence of ADR-2 and ADR-8, and is the most likely driver of a future MAJOR revision.
- **Real images break strict single-file purity.** Once real artwork is added via `src`, the site comprises `index.html` plus an `assets/` folder. The HTML/CSS/JS remains one file, which is the intent of NFR-01, but the user should understand the distinction.
- **Font dependency.** Offline or with the CDN blocked, the page renders in system fonts. The typographic identity is materially weaker in that state, though layout is preserved via `size-adjust`.
- **Browser floor.** Safari < 15.4 and Firefox < 98 lose the lightbox (fallback to anchored captions); Firefox < 103 loses glass translucency. These are accepted under A4.
- **No automated behavioural tests.** Motion and keyboard behaviour are verified manually against the checklist. Regressions between segments depend on the Tester re-running affected rows; ADR-9 records the option to add Playwright as tooling later.
- **Performance is device-relative.** 60 fps is verified under a defined protocol (A10), not guaranteed on every device. Low-end mobile devices are excluded from motion by the coarse-pointer gate,

which also removes the most expensive effects.

- **Single accent colour and two type families.** The design system deliberately offers few knobs. Richer theming (multiple accents, per-section palettes) would need additional token groups (MINOR revision).
- **No SEO, analytics or telemetry.** Explicit non-objectives; the site is discoverable only through direct links and basic HTML semantics.
- **Sequential development.** Because the deliverable is one file, segments cannot be developed in parallel. The roadmap's twelve segments are therefore executed strictly in order (P2/P3 may be swapped, not overlapped).

27.3 Open questions for the Collaborator

1. Confirm A1–A10, especially the font CDN (A7) and default theme (A8).
2. Decide whether the "Could" requirements FR-14 (navigation) and NFR-10 (150 KB) are in scope for v1 or deferred.
3. Confirm the P2-before-P3 ordering (ADR-10) or request the lightbox first.
4. Provide the artist's actual name, statement and contact details if available, so S03 can use real copy instead of placeholders.